



AGILE INTEGRITY INDEX

VALUES, PRACTICES, AND ANTI-PATTERNS IN MODERN SCRUM TEAMS

TYLER BAO

SENG 6235 - SOFTWARE PROJECT MANAGEMENT

EAST CAROLINA UNIVERSITY

TABLE OF CONTENTS

ABSTRACT	7
SECTION 1: INTRODUCTION	7
SECTION 2: LITERATURE REVIEW	8
2.1 THE AGILE MANIFESTO	9
2.2 THE SCRUM FRAMEWORK	9
2.3 SCRUM ANTI-PATTERNS	10
2.4 EXISTING AGILE MEASUREMENT FRAMEWORKS	10
2.5 GAP IN THE LITERATURE	10
SECTION 3: METHODOLOGY	11
3.1 RESEARCH DESIGN	11
3.2 DATA SOURCES	11
3.2.1 ECU Pirate Forge Discord	11
3.2.2 ECU Pirate Forge GitHub	12
3.2.3 Sprint Review Observation	12
3.2.4 Sprint Artifacts	12
3.3 AI-ASSISTED DATA COLLECTION AND SCORING	13
3.4 DATA COLLECTION PROCEDURES	13
Phase 1 - Document Collection	13
Phase 2 - Sprint Review Observation	13
Phase 3 - Cross-Source Synthesis	14
3.5 SCORING PROCEDURES	14
3.6 LIMITATIONS	14
SECTION 4: AGILE INTEGRITY INDEX (AII) MODEL DESIGN	15
4.1 RATIONALE FOR A COMPOSITE INDEX	15
4.2 MODEL OVERVIEW AND WEIGHTING	15
Table 1: All Dimension Weighting and Scoring Model	15
Composite Formula:	16
Weighting Justification:	16
4.3 DIMENSION A: AGILE MANIFESTO VALUE ALIGNMENT (25%)	17
4.3.1 Individuals and Interactions Over Processes and Tools	17
4.3.2 Working Software Over Comprehensive Documentation	17
4.3.3 Customer Collaboration Over Contract Negotiation	18
4.3.4 Responding to Change Over Following a Plan	18

4.4 DIMENSION B: SCRUM BEST-PRACTICE ADHERENCE (45%)	19
4.4.1 <i>Sprint Planning</i>	19
4.4.2 <i>Daily Scrum</i>	19
4.4.3 <i>Sprint Review</i>	20
4.4.4 <i>Sprint Retrospective</i>	20
4.4.5 <i>Backlog Refinement</i>	20
4.4.6 <i>Definition of Done</i>	20
4.4.7 <i>Transparency, Inspection, and Adaptation</i>	21
4.5 DIMENSION C: ANTI-PATTERN FREQUENCY (30%).....	21
4.5.1 <i>Anti-Pattern Detection Methods</i>	21
Table 2: Dimension C Anti-Pattern Penalty Schedule and Detection Methods.....	22
4.5.2 <i>Penalty Weighting Rationale</i>	22
4.6 ALL SCORE CALCULATION	23
SECTION 5: DATA COLLECTION AND ANALYSIS	24
5.1 SCRUMPILOT	24
5.2 OBSERVED TEAM. ALL ANALYSIS	25
<i>FormFlow</i>	25
<i>Sunflower Tracker</i>	27
<i>Pulse</i>	27
5.3 SPRINT-BY-SPRINT ALL SCORES	28
Table 3: Composite All Scores by Team	29
SECTION 6: FINDINGS	29
6.1 VALUE ALIGNMENT CORRELATIONS	29
6.2 HIGH-IMPACT SCRUM PRACTICES	30
6.3 MOST HARMFUL ANTI-PATTERNS	30
6.4 ALL AS A PREDICTIVE INDICATOR.....	31
6.5 CROSS-TEAM COMPARISON.....	32
SECTION 7: VISUALIZATION DASHBOARD	32
7.1 ALL TREND LINE	32
7.2 ANTI-PATTERN HEATMAP.....	32
7.3 VALUE-ALIGNMENT RADAR CHART	33
7.4 SCRUM PRACTICE ADHERENCE SCORECARD.....	33
7.5 ADDITIONAL VISUALIZATIONS	33
SECTION 8: RECOMMENDATIONS.....	33

8.1 STRENGTHENING AGILE VALUE ALIGNMENT	33
8.2 IMPROVING SCRUM PRACTICE EXECUTION	34
8.3 ELIMINATING OR MITIGATING ANTI-PATTERNS.....	34
8.4 ENHANCING PREDICTABILITY AND QUALITY	35
SECTION 9: REFLECTION.....	35
9.1 SURPRISES AND UNEXPECTED FINDINGS	35
9.2 LIMITATIONS OF THE AII MODEL.....	36
9.3 LIMITATIONS OF THE APPLIED RESEARCH	36
9.4 PERSONAL TAKEAWAYS.....	36
SECTION 10: CONCLUDING THOUGHTS.....	37
METHODOLOGY BIAS AND LIMITATIONS.....	37
CONCLUSION	38
REFERENCES	39
APPENDIX A: AGILE MANIFESTO	40
A.1 ORIGINS AND CONTEXT.....	40
A.2 THE FOUR VALUES.....	40
<i>Value 1: Individuals and Interactions Over Processes and Tools.....</i>	<i>41</i>
<i>Value 2: Working Software Over Comprehensive Documentation.....</i>	<i>41</i>
<i>Value 3: Customer Collaboration Over Contract Negotiation</i>	<i>41</i>
<i>Value 4: Responding to Change Over Following a Plan.....</i>	<i>42</i>
A.3 THE TWELVE PRINCIPLES.....	42
<i>Delivery and Value.....</i>	<i>42</i>
<i>Embracing Change</i>	<i>42</i>
<i>Collaboration and Communication</i>	<i>43</i>
<i>Technical Excellence</i>	<i>43</i>
<i>Team Dynamics and Improvement.....</i>	<i>43</i>
A.4 APPLICATION TO THE AII	44
APPENDIX B: SCRUM BEST PRACTICES	44
B.1 OVERVIEW OF THE SCRUM FRAMEWORK.....	44
B.2 SCRUM VALUES.....	45
<i>B.2.1 Commitment</i>	<i>45</i>
<i>B.2.2 Focus</i>	<i>45</i>
<i>B.2.3 Openness.....</i>	<i>45</i>

<i>B.2.4 Respect</i>	46
<i>B.2.5 Courage</i>	46
B.3 SCRUM ACCOUNTABILITIES	46
<i>B.3.1 The Product Owner</i>	46
<i>B.3.2 The Scrum Master</i>	47
<i>B.3.3 The Developers</i>	47
B.4 SCRUM EVENTS	48
<i>B.4.1 The Sprint</i>	48
<i>B.4.2 Sprint Planning</i>	48
<i>B.4.3 Daily Scrum</i>	49
<i>B.4.4 Sprint Review</i>	49
<i>B.4.5 Sprint Retrospective</i>	50
<i>B.4.6 Backlog Refinement</i>	50
B.5 SCRUM ARTIFACTS	50
<i>B.5.1 Product Backlog</i>	50
<i>B.5.2 Sprint Backlog</i>	51
<i>B.5.3 Increment</i>	51
<i>B.5.4 Definition of Done</i>	52
APPENDIX C: COMPREHENSIVE SCRUM ANTI-PATTERN LIST	52
C.1 INTRODUCTION TO SCRUM ANTI-PATTERNS	52
C.2 PROCESS ANTI-PATTERNS	53
<i>C.2.1 Zombie Scrum (All Penalty: -15)</i>	53
<i>C.2.2 No Real Definition of Done (All Penalty: -15)</i>	54
<i>C.2.3 Carryover Work (All Penalty: -10)</i>	54
<i>C.2.4 Skipped or Perfunctory Ceremonies</i>	55
<i>C.2.5 ScrumBut (All Penalty: -8)</i>	55
<i>C.2.6 Fake Retrospectives</i>	56
<i>C.2.7 Overloaded Sprints (All Penalty: -10)</i>	56
C.3 ROLE ANTI-PATTERNS	57
<i>C.3.1 Product Owner as Project Manager (All Penalty: -10)</i>	57
<i>C.3.2 Absent Product Owner</i>	58
<i>C.3.3 Scrum Master as Note-Taker or Administrator</i>	58
<i>C.3.4 Developer Silos</i>	58
<i>C.3.5 Hero Developer</i>	59
C.4 COMMUNICATION ANTI-PATTERNS	59

<i>C.4.1 Daily Scrum as Status Report (All Penalty: -10)</i>	59
<i>C.4.2 Impediment Hoarding</i>	60
<i>C.4.3 HiPPO-Driven Backlog</i>	60
C.5 PLANNING ANTI-PATTERNS	60
<i>C.5.1 Velocity Gaming</i>	60
<i>C.5.2 No Sprint Goal</i>	61
<i>C.5.3 Estimation Anchoring</i>	61
<i>C.5.4 Sprint Zero Abuse</i>	61
C.6 CULTURAL ANTI-PATTERNS	62
<i>C.6.1 Multitasking Across Projects (All Penalty: -8)</i>	62
<i>C.6.2 Forced Agile</i>	62
<i>C.6.3 Lack of Psychological Safety</i>	63
<i>C.6.4 Blame Culture</i>	63
<i>C.6.5 Management by Jira</i>	63
C.7 ANTI-PATTERN SUMMARY TABLE	64
APPENDIX D: ALL SCORING RUBRIC	65
DIMENSION A RUBRIC: AGILE MANIFESTO VALUE ALIGNMENT	65
DIMENSION B RUBRIC: SCRUM BEST-PRACTICE ADHERENCE	65
DIMENSION C RUBRIC: ANTI-PATTERN FREQUENCY	66
ALL COMPOSITE SCORE INTERPRETATION	67

ABSTRACT

This paper presents the Agile Integrity Index (All), a composite scoring model designed to measure Agile team health across three distinct dimensions: Agile Manifesto value alignment, Scrum best-practice adherence, and anti-pattern frequency. The model was applied to four student Scrum teams. ScrumPilot, FormFlow, Pulse, and Sunflower Tracker, operating within the ECU Pirate Forge environment during the Spring 2026 semester.

A deliberate methodological choice shaped this research from the start. Because I was a member of one of the teams being evaluated, personal bias was a genuine risk. To best address it, I chose to ground the analysis almost entirely in hard data. GitHub repository records, Discord communication logs, and team-authored process documents. The remaining analysis which required my direct input was sprint review observation, which is supported with transcript logs of the reviews themselves.

An AI model was used to assist in processing the volume of data and applying my scoring rubric systematically. Every AI-generated finding was reviewed, challenged, and corrected only where necessary. The goal was a scoring process that could not easily be accused of favoritism, even when the researcher could be accused of having a rooting interest in the outcome.

The results revealed a 50-point gap between the highest and lowest scoring teams, with ScrumPilot scoring 97 (Exemplary) and Pulse scoring 45 (At Risk). The most significant finding was not the scores themselves, but rather what produced them since all four teams documented nearly identical process standards. The difference was entirely behavioral, based solely in whether teams actually practiced what they documented. The All framework is designed to be replicable by future cohorts and is accompanied by a visualization dashboard and a full scoring rubric in the appendices which were built with AI assistance in order to fully convey the details of the Agile Manifesto, Scrum best practices, and Scrum Anti-Pattern utilized in designing this scoring model.

SECTION 1: INTRODUCTION

There is a meaningful difference between a team that does Scrum and a team that practices it. The former holds the ceremonies, maintains the board, and calls the daily standup a Daily Scrum. The latter does those things, but with a genuine understanding of why each element exists and embraces the process for the benefits it offers, while maintaining a willingness to adapt when the process stops serving the work. That distinction is harder to measure than velocity or sprint completion rate, and that difficulty was a motivation of this research. Every student was required to participate in Scrum, but how well could each team embrace the practice and generate meaningful results from it.

I am a graduate student in SENG 6235 – Software Project Management at East Carolina University, embedded in the ScrumPilot team within the ECU Pirate Forge cohort, a group of four student teams building real software products across a semester long Scrum project. Each team had access to the same tools: a Discord server as their primary communication channel, GitHub for source control and issue tracking, and the same course requirements. By the end of four sprints, the teams had diverged dramatically, not in the tools they used, but in how genuinely they utilized them. That divergence is the raw material for this study.

The core problem is diagnostic. Existing Agile measurement tools tell you what happened but not why. Velocity, burndown charts, and sprint completion rates measure a team's output, but they cannot tell you whether a team is executing Scrum with genuine intent or just going through the motions. As we will cover, a team can sustain impressive velocity while systematically violating Agile values or document a thorough Definition of Done while never actually enforcing it. What if a framework simultaneously evaluated value alignment, practice adherence, and anti-pattern presence in a single diagnostic score? The Agile Integrity Index is an attempt to achieve just that, filling a gap left by more widely used frameworks.

The central research question this paper addresses is: ***How do Agile Manifesto values, Scrum best-practice adherence, and the presence or absence of anti-patterns collectively influence team outcomes?*** Four sub-questions ultimately guide the analysis:

- Which Agile values correlate most strongly with positive outcomes?
- Which Scrum practices have the highest impact on predictability or quality?
- Which anti-patterns are most harmful and how can they be detected early?
- Can a composite index like AI reliably predict sprint success?

The study draws on our four teams and comparatively analyzes each using all available Discord, GitHub, and observation data across the full Spring 2026 semester.

SECTION 2: LITERATURE REVIEW

This section establishes the theoretical and empirical foundation across the three bodies of knowledge that were used collectively to define the AI model:

- The Agile Manifesto and its behavioral implications
- The Scrum framework and its prescribed practices

- Scrum anti-patterns

The section concludes with identification of the measurement gap this research addresses.

2.1 THE AGILE MANIFESTO

The Agile Manifesto was authored in February 2001 by seventeen software practitioners. The document established a philosophical orientation, four value preferences and twelve supporting principles, rather than a prescriptive methodology. Its central insight was that software development is a knowledge generating activity, not a manufacturing process, and that the practices governing it should reflect that distinction. The four values are expressed as deliberate preferences between two legitimate goods, with the caveat that "while there is value in the items on the right, we value the items on the left more" (Beck et al., 2001). This framing is frequently misread as an absolute rejection of documentation, contracts, or planning. This is a misinterpretation with observable consequences in teams that abandon process discipline in the name of being Agile. The twelve principles elaborate on these values, with Principle 12, "At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly" (Beck et al., 2001), providing the philosophical foundation for the Sprint Retrospective and for the AI's scoring of continuous improvement. The gap between Manifesto intent and real-world adoption has been a persistent subject of academic inquiry, with researchers consistently finding that teams adopt Agile vocabulary and ceremonies while retaining pre-Agile management behaviors and decision-making patterns.

2.2 THE SCRUM FRAMEWORK

"Scrum is a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems" (Schwaber & Sutherland, 2020). The Scrum Guide (Schwaber & Sutherland, 2020) is the canonical reference for the Scrum framework and the primary source for Dimension B of the AI. The 2020 revision is notable for replacing the term 'roles' with 'accountabilities,' for elevating psychological safety as a prerequisite condition rather than a cultural aspiration, and for strengthening the Sprint Goal as the primary unit of sprint commitment. The Guide defines Scrum as "a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems" and grounds it in three empirical pillars, transparency, inspection, and adaptation, that are interdependent by design. The AI operationalizes all three pillars as a combined scoring indicator in Dimension B's Transparency/Inspection/Adaptation row. Empirical research on Scrum effectiveness has consistently found that adoption is widespread but quality of implementation varies dramatically: teams that practice the ceremonies without the values tend to experience the costs of Scrum's overhead without accessing its benefits (Dikert, Paasivaara, & Lassenius, 2016).

2.3 SCRUM ANTI-PATTERNS

The concept of anti-patterns originates with Koenig (1995) and was expanded by Brown et al. (1998), who defined them as solutions that appear reasonable on the surface but consistently produce outcomes worse than having no solution at all. In the Scrum context, Russo and Verwijs (2021) provided the most influential contemporary taxonomy through their work on Zombie Scrum, teams that perform all ceremonies without any of their intent. Their research identified organizational conditions (lack of genuine autonomy, disconnection from product outcomes, management behaviors that contradict Scrum principles) as the root causes of Scrum dysfunction rather than team incompetence. Edmondson's (1999) foundational work on psychological safety is particularly relevant to cultural anti-patterns: she defines it as "a shared belief held by members of a team that the team is safe for interpersonal risk taking" and demonstrates its relationship to team learning behavior. Google's Project Aristotle (Duhigg, 2016) later identified psychological safety as the single most important factor distinguishing high-performing teams, a finding directly observable in this study's comparison of ScrumPilot's open accountability culture against Pulse's externally coerced compliance.

2.4 EXISTING AGILE MEASUREMENT FRAMEWORKS

Several established approaches exist for measuring Agile team performance. Velocity tracks story points completed per sprint, providing a throughput baseline but no diagnostic capability, a team can game velocity by inflating estimates or reduce it by under-committing. Sprint goal achievement rates provide binary success/failure signals per sprint but do not explain the drivers of failure. The Spotify Health Check model (Kniberg & Ivarsson, 2012) uses team self-assessment across dimensions including ease of release, fun, and suitable process, but relies on subjective team reporting with no external validation. Cumulative Flow Diagrams and cycle time analytics provide flow efficiency signals but require mature tooling and discipline to maintain. None of these frameworks synthesize value alignment, practice adherence, and anti-pattern presence into a single composite diagnostic score. Each captures part of the picture; none provides the integrated view that the AI is designed to offer.

2.5 GAP IN THE LITERATURE

The literature reveals a diagnostic gap in that existing Agile metrics measure outputs like velocity and completion rate, or inputs such as ceremony participation and backlog health, but not the relationship between them. A team that scores well on velocity may be gaming estimates. A team that holds all ceremonies may be practicing Zombie Scrum. No widely adopted framework asks simultaneously whether team behaviors reflect genuine Agile values? Do ceremonies serve their intended purpose? Are active dysfunctions degrading performance? The Agile Integrity Index addresses this gap by

synthesizing all three dimensions into a composite sprint-level score that enables both trend analysis within a team and comparative analysis across teams.

SECTION 3: METHODOLOGY

This study employs a mixed method case study design to apply the Agile Integrity Index across four student Scrum teams in the ECU Pirate Forge environment. Quantitative scoring was derived directly from GitHub activity metric such as PR counts, total commits, and issue tracking data. Qualitative analysis informed scoring decisions for Dimension A values and cultural anti-patterns, drawing on Discord communications, sprint review observations, and process documents authored by each of the teams such as Definition of Ready, Definition of Done, and Git Workflow documentation. The unit of analysis is the team across the full four sprint observation period, with the aggregate All scores representing overall team performance rather than per-sprint snapshots.

3.1 RESEARCH DESIGN

The mixed-methods design was selected because Agile integrity cannot be fully assessed through quantitative data alone. GitHub commit counts measure activity but not quality. PR counts measure process throughput but not review substance. Ceremony frequency measures compliance but not intent. Qualitative observation and communication analysis provide a contextual layer that makes quantitative evidence interpretable. For example, ScrumPilot's 91 closed PRs is a strong quantitative signal, but its meaning is ultimately confirmed by the qualitative evidence of substantive multi-reviewer code discussions. Pulse's 56 closed PRs look comparable on the surface, but qualitative Discord evidence reveals professor directed process compliance rather than self-organization by the team.

3.2 DATA SOURCES

3.2.1 ECU PIRATE FORGE DISCORD

The ECU Pirate Forge Discord server contains dedicated channels for each team, including standup, planning, retrospective, general, diagram, and ceremony-specific channels. All team channels were reviewed across the full observation period. Analysis coded for: message volume per channel, message initiators (who started conversations vs. who responded), impediment surfacing (explicit mention of blockers), ceremony evidence (standup content, retro outputs, planning discussions), and communication directionality (peer-to-peer vs. reporting to instructor). Where teams used additional communication channels (such as in-class meetings documented in sprint summaries), this supplementary evidence was incorporated. The limitations here are with any of this communication not explicitly detailed within either

Discord or GitHub, which I would be blind to in the analysis. The ScrumPilot team's Discord bot (Scrum Lord) generated automated transcriptions and sprint summaries across all teams, providing an additional structured data source.

3.2.2 ECU PIRATE FORGE GITHUB

GitHub repository data was collected from four public repositories in the ECU-Pirate-Forge organization. Metrics collected included: total commits to the main branch (as an indicator of sprint-end promotion discipline), total commits to the development branch to identify whether the team's DoD was followed, total closed pull requests (as an indicator of working software throughput), open issues at project end (analyzed for whether open items represent future backlog vs. unresolved commitments), open PRs at project end (as an indicator of incomplete work), and contributor distribution (as an indicator of cross-member collaboration vs. developer silos). Branch naming conventions and git workflow adherence were assessed by comparing documented workflows against observed commit patterns. The main vs. development branch distinction proved particularly informative. All four teams documented the same sprint-end dev to main promotion process in their DoD, making the main branch commit count a direct behavioral indicator of workflow adherence.

3.2.3 SPRINT REVIEW OBSERVATION

Direct sprint review observations were conducted for the Sunflower Tracker team and the FormFlow team during Sprint 4. Observations were structured around six indicators:

1. Whether a sprint goal was articulated at the start of the review
2. Whether a Scrum board was displayed
3. Whether PBIs were identified before demos
4. Whether working software was demonstrated
5. Whether stakeholder feedback was solicited and received
6. Whether the team or the professor drove the session.

Field notes were taken in real time and cross-referenced against sprint summary documentation and GitHub state at the time of observation.

3.2.4 SPRINT ARTIFACTS

Sprint artifacts available for analysis included Bot generated sprint summaries (provided for all four teams across all four sprints), Definition of Done documents, Definition of Ready documents, and git workflow documentation. Burndown

charts and formal velocity tracking were only consistently documented for ScrumPilot, which recorded sprint velocity beginning in Sprint 3. Other teams did not provide burndown artifacts for analysis, which is itself a finding for Dimension B Transparency/Inspection/Adaptation scoring.

3.3 AI-ASSISTED DATA COLLECTION AND SCORING

This study leveraged three technology-assisted approaches to support data collection and analysis at a scale that would not have been feasible through manual review alone.

GitHub's public repository data provided quantitative artifact evidence including commit history, pull request counts, issue tracking, and branch activity across all four teams. Discord channel histories provided the primary source of communication and ceremony evidence. The ScrumPilot team's own Discord bot, ScrumLord, generated automated meeting transcriptions and sprint summaries across all teams, which served as a foundational data source for ceremony quality assessment.

An AI model was used to assist in reviewing collected evidence against the All scoring rubric, identifying patterns across large volumes of Discord and GitHub data in order to draft initial score recommendations across all three dimensions. The researcher acted as a critical guardrail throughout this process, correcting scoring assumptions that did not account for missing context or fell out of scope, challenging interpretations that appeared unfair or unsupported by evidence, and making all final scoring decisions independently.

This hybrid approach of AI-assisted pattern recognition with human-controlled scoring decisions, is itself a model worth noting for future cohorts. It demonstrates that the All framework can be applied efficiently at scale while maintaining fairness, and without sacrificing the contextual judgment that purely automated scoring cannot provide.

3.4 DATA COLLECTION PROCEDURES

Data collection followed a three phase procedure replicable for future cohorts.

PHASE 1 - DOCUMENT COLLECTION

All four team repositories were accessed for their Discord-posted process documents (DoD, DoR, git workflow) and GitHub repository data. Sprint summary documents were collected covering all four sprints for all four teams.

PHASE 2 - SPRINT REVIEW OBSERVATION

Direct observation was conducted for two teams (Sunflower Tracker and FormFlow) using the six-indicator protocol described in 3.2.3 Sprint Review Observation. The Pulse team review was missed as the schedule was not added to the Events schedule in the Pirate Forge Discord server.

PHASE 3 - CROSS-SOURCE SYNTHESIS

Discord channel histories were reviewed for each team, cross-referencing with GitHub data to identify patterns. Specifically: message volume trends, standup content and directionality, impediment mentions, and evidence of ceremony occurrence. AI-assisted pattern recognition (Section 3.3) was applied during this phase to process the volume of Discord and GitHub data, with human review and correction of all preliminary findings.

3.5 SCORING PROCEDURES

Scoring followed the rubric in [Appendix D](#) for all three dimensions.

For [Dimension A](#), each of the four Agile values was assessed by gathering all available evidence for a team (Discord, GitHub, sprint summaries, observation) and mapping it to the 1–5 behavioral anchors in the rubric.

For [Dimension B](#), each of the seven practice areas was scored by identifying the strongest and weakest evidence available and selecting the score level whose description best matched the observed pattern. For ambiguous evidence, where behaviors fit between two levels, the lower score was assigned to maintain conservative rigor.

For [Dimension C](#), each anti-pattern was evaluated against its specific evidence threshold (Appendix D). Anti-patterns were penalized only when the threshold was clearly met; qualitative concern without meeting the threshold was noted but not penalized.

3.6 LIMITATIONS

Four limitations warrant acknowledgment.

1. Single-researcher scoring: with no inter-rater reliability check, scoring decisions reflect one researcher's judgment. Conservative scoring thresholds and explicit evidence documentation mitigate but do not eliminate this limitation.
2. Student project context: findings may not generalize to industry teams operating under different incentive structures, organizational pressures, and consequence models.

- 3. Discord as a proxy: if significant communication occurred in other channels (text messages, in-person conversations, Microsoft Teams), it is not captured in this analysis. Sprint summaries for some teams reference in-class meetings that were not available for direct analysis.
- 4. GitHub main branch as a proxy: the main branch commit count is a function of the team's sprint-end promotion discipline, not a direct measure of all code written. The development branch analysis confirmed that substantially more work existed in dev branches than main for all teams.

SECTION 4: AGILE INTEGRITY INDEX (AII) MODEL DESIGN

4.1 RATIONALE FOR A COMPOSITE INDEX

Existing approaches to measuring Agile team performance tend to focus on a single dimension. Velocity tracks output. Predictability indices compare planned to completed work. Sprint goal achievement rates measure delivery consistency. While each of these metrics provides value in isolation, none of them can distinguish why a team is underperforming. A team may sustain strong velocity while systematically violating Agile values, executing ceremonies mechanically without genuine inspection or adaptation. Conversely, a team deeply aligned with Agile values may still produce inconsistent results due to poor Scrum practice execution or persistent anti-patterns that erode throughput over time.

The Agile Integrity Index (AII) is designed to address this diagnostic gap by synthesizing three distinct dimensions into a single composite score: Agile Manifesto value alignment, Scrum best-practice adherence, and anti-pattern frequency. Each dimension measures something the others cannot. Together they produce a 0-100 score that reflects the full picture of how a team operates, not just how productive it appears.

The AII produces a score between 0 and 100 based on the performance across all four sprints, enabling trend analysis over time and comparison across teams. A declining AII score is an early warning signal regardless of whether velocity appears stable. A rising AII score indicates that the team is not only delivering but doing so in a manner consistent with Agile principles, a distinction that matters for long-term sustainability.

4.2 MODEL OVERVIEW AND WEIGHTING

The AII is composed of three weighted dimensions:

TABLE 1: AII DIMENSION WEIGHTING AND SCORING MODEL

Dimension	Weight	Description
A - Agile Manifesto Value Alignment	25%	Measures how well the team embodies the four Agile values in observable behavior
B - Scrum Best Practice Adherence	45%	Measures consistency and quality of Scrum event and artifact execution
C - Anti-Pattern Frequency	30%	Inverted penalty score reflecting the absence of detected Scrum anti-patterns

COMPOSITE FORMULA:

$$All = (0.25 \times A) + (0.45 \times B) + (0.30 \times C)$$

All three dimensions normalize to a 0-100 scale. Dimension C is inverted: it begins at 100 and penalties are subtracted for each detected anti-pattern, with a floor of zero. This reflects the asymmetry between good practices and active dysfunctions. Anti-patterns do not simply represent the absence of something good. They actively degrade performance.

WEIGHTING JUSTIFICATION:

Dimension B carries the highest weight at 45% because Scrum practices are the most directly observable dimension available from the data sources used in this study. Discord activity, GitHub artifacts, and ceremony observation all map cleanly to specific Scrum events and artifacts. Practices are also where teams have the most immediate agency for change. A team can restructure its Daily Scrum tomorrow in a way that it cannot immediately restructure its cultural values.

Dimension C carries 30% because anti-patterns are actively destructive rather than merely absent goods. A team that scores moderately on value alignment and practice adherence can still have its performance severely degraded by one persistent anti-pattern. The penalty based inversion model reflects this asymmetry.

Dimension A carries 25% not because values are unimportant, in fact they are foundational. The lower weight reflects the difficulty of measuring them objectively from Discord and GitHub alone. Value alignment is primarily assessed through

qualitative behavioral indicators, which introduces more subjectivity than the other two dimensions. I weighted it conservatively to reflect that reality, not to minimize its importance.

4.3 DIMENSION A: AGILE MANIFESTO VALUE ALIGNMENT (25%)

The Agile Manifesto articulates four core values that distinguish Agile development from plan driven approaches. These values do not prescribe specific practices. Instead, they establish a philosophical orientation that should shape how a team makes decisions. Evaluating value alignment therefore requires identifying observable patterns of action that are consistent or inconsistent with each value.

Each of the four values is scored on a 1–5 scale across the observation period. The four scores are averaged and multiplied by 20 to normalize to our 0–100 range.

A Score = (Average of four value scores) × 20

Full scoring criteria for each level are provided in [Appendix D](#).

4.3.1 INDIVIDUALS AND INTERACTIONS OVER PROCESSES AND TOOLS

This value emphasizes human collaboration as the primary driver of software development success. Teams that embody this value prioritize direct communication, cross functional problem solving, and relationship building over rigid adherence to process or tools.

Observable indicators include the frequency and depth of team communication in Discord, evidence of team members reaching out to each other proactively rather than waiting for scheduled ceremonies, the depth of engagement in pull request reviews, and the degree to which GitHub comments reflect genuine technical dialogue versus transactional approval.

Low scores in this dimension may manifest as minimal Discord activity outside of structured ceremonies, pull requests approved without substantive review, or communication patterns where one or two members dominate while others are largely silent.

4.3.2 WORKING SOFTWARE OVER COMPREHENSIVE DOCUMENTATION

This value holds that the primary measure of progress is software that functions, not plans, reports, status updates, or documentation. Teams that embody this value organize their effort around producing demonstrable increments and treat everything else as secondary.

Observable indicators include the frequency of commits and merges per sprint, whether sprint reviews include working demonstrations rather than progress reports, the ratio of completed to planned stories, and evidence that the team's Definition of Done includes functional and quality criteria rather than documentation checkboxes alone.

Low scores in this dimension may appear as sprints ending without a demonstrable increment, a pattern of stories reaching "in progress" status but not crossing the Definition of Done, or sprint reviews that describe work done rather than demonstrate it.

4.3.3 CUSTOMER COLLABORATION OVER CONTRACT NEGOTIATION

This value emphasizes ongoing engagement with the customer or stakeholder as a partner in the development process, rather than a party whose requirements are fixed at the outset and evaluated at delivery. In a student project context, "customer collaboration" is interpreted broadly to include stakeholder engagement during sprint reviews, instructor feedback integration, and team responsiveness to feedback received. (Beck et al., 2001)

Observable indicators include stakeholder presence or engagement during sprint reviews, evidence of backlog adjustments driven by feedback, and the degree to which the team treats the product backlog as a living document shaped by ongoing input rather than a fixed specification.

Low scores in this dimension may appear as sprint reviews conducted without meaningful stakeholder engagement, backlogs that do not evolve across sprints, or feedback that is acknowledged but not acted upon.

4.3.4 RESPONDING TO CHANGE OVER FOLLOWING A PLAN

This value recognizes that software development is inherently uncertain and that teams must be capable of adapting to new information, shifting priorities, and evolving understanding. Teams that embody this value treat change as expected rather than exceptional and build adaptability into their process.

Observable indicators include instances of mid sprint backlog refinement in response to new information, retrospective action items that produce visible changes in subsequent sprints, and team behavior when impediments or surprises arise, specifically whether the team adapts the plan or forces the reality to fit the plan.

Low scores in this dimension may appear as repeated carryover of the same items without adjustment, retrospectives that produce no observable change in behavior, or teams that respond to unexpected complexity by extending hours rather than re-scoping.

4.4 DIMENSION B: SCRUM BEST-PRACTICE ADHERENCE (45%)

The Scrum framework, as defined by the Scrum Guide (Schwaber & Sutherland, 2020), prescribes a specific set of events, artifacts, and accountabilities. The second dimension of the All evaluates how consistently and meaningfully a team implements these elements. Consistency alone is insufficient, a team can hold a Daily Scrum every day while conducting it as a status meeting, which satisfies the letter of the practice while violating its intent.

Seven practice areas are scored on a 1–5 scale. The seven scores are averaged and multiplied by 20 to normalize to a 0–100 range.

B Score = (Average of seven practice scores) × 20

Full scoring criteria for each level are provided in [Appendix D](#). A reference to Scrum best-practice standards is available in [Appendix B](#).

4.4.1 SPRINT PLANNING

Effective sprint planning establishes a shared sprint goal, selects backlog items the team has confidence in completing, and results in a sprint backlog that reflects a realistic understanding of team capacity. The sprint goal is particularly important, it provides coherence to the sprint and enables the team to make trade-off decisions if conditions change mid-sprint.

Indicators of strong sprint planning include a clearly stated sprint goal visible to all team members, evidence that stories were estimated before selection, capacity awareness reflected in the sprint backlog size, and planning discussions visible in Discord or captured in meeting notes.

4.4.2 DAILY SCRUM

The Daily Scrum is a 15 minute event designed to inspect progress toward the sprint goal and adapt the sprint backlog as needed. Its value comes from creating a daily opportunity for the team to synchronize and surface impediments early. The most common dysfunction of this event is its transformation into a status reporting mechanism, team members reporting to the Scrum Master or a manager rather than coordinating with each other toward the sprint goal.

Indicators of an effective Daily Scrum include message patterns in Discord that reflect peer-to-peer coordination, evidence of impediments being surfaced and addressed within a short timeframe, and daily standup content focused on sprint goal progress rather than task-level status.

4.4.3 SPRINT REVIEW

The sprint review is the team's primary opportunity to inspect the increment and adapt the product backlog based on stakeholder feedback. A meaningful sprint review demonstrates working software, invites genuine feedback, and results in backlog adjustments that reflect what was learned.

Indicators include evidence that a demonstration occurred, stakeholder engagement or feedback captured in Discord or meeting notes, and backlog changes following the review that reflect the feedback received.

4.4.4 SPRINT RETROSPECTIVE

The retrospective is the team's mechanism for continuous process improvement. It provides a dedicated space to inspect how the team worked during the sprint and to identify actionable adjustments. Its effectiveness depends on two factors: the quality of the issues identified, and whether previous action items were actually implemented.

Indicators include evidence that a retrospective occurred each sprint, the creation of specific and actionable improvement items, and follow-through on those items in subsequent sprints. Repeated retrospective action items that never close are a strong signal of dysfunction.

4.4.5 BACKLOG REFINEMENT

Backlog refinement ensures that upcoming stories are well-understood, appropriately sized, and ready to be selected during sprint planning. Without regular refinement, sprint planning becomes inefficient and teams frequently discover mid-sprint that stories are larger or less understood than assumed.

Indicators include evidence of refinement activity between sprints in Discord or GitHub, stories entering the sprint with acceptance criteria already defined, and story sizes that suggest thoughtful decomposition rather than large undivided items.

4.4.6 DEFINITION OF DONE

The Definition of Done (DoD) is the shared standard that defines when a product backlog item is considered complete. Without a meaningful DoD, teams risk accumulating technical debt, shipping low quality increments, or completing stories by definition while leaving work practically unfinished.

Indicators include evidence that a written DoD exists, that it is referenced during sprint activities, and that completed stories consistently meet its criteria. A DoD that includes only "code committed" without quality, testing, or review criteria is scored lower than one that reflects genuine quality standards.

4.4.7 TRANSPARENCY, INSPECTION, AND ADAPTATION

The three pillars of empiricism underline the entire Scrum framework. A team operating with true transparency keeps its board current, shares impediments openly, and makes its metrics visible. Inspection requires that the team actually examines these artifacts regularly. Adaptation requires that what is learned during inspection produces change.

Indicators include a GitHub or project board that reflects the current state of work, metrics that are tracked and discussed, and evidence that the team changes its behavior in response to what it observes rather than continuing patterns that are not working.

4.5 DIMENSION C: ANTI-PATTERN FREQUENCY (30%)

The third dimension of the All takes an inverted approach. Rather than scoring the presence of desirable behaviors, it scores the *absence* of known dysfunctions. Anti-patterns in Scrum are recurring patterns of behavior that appear reasonable or even intentional on the surface but consistently produce negative outcomes. Because anti-patterns actively degrade team performance, their presence warrants a penalty-based model rather than a simple absence-of-good-practices deduction.

Dimension C begins at 100 each sprint. Penalties are subtracted for each anti-pattern detected, with a floor of zero.

C Score = 100 – Σ (penalties for detected anti-patterns), minimum 0

Each anti-pattern carries a penalty weight that reflects its severity and the scope of its negative impact on team outcomes. Anti-patterns are classified as High (–15), Medium (–10), or Low (–8) severity. A full list of anti-patterns and their definitions is provided in [Appendix C](#).

4.5.1 ANTI-PATTERN DETECTION METHODS

Anti-patterns are detected through systematic review of available data sources. Discord message patterns reveal communication dysfunctions such as status reporting in standups or absence from ceremonies. GitHub activity reveals delivery related anti-patterns such as carryover work, overloaded sprints, and lack of Definition of Done compliance.

Sprint review observations reveal role-based anti-patterns and evidence of Zombie Scrum. The following table summarizes the anti-patterns tracked in this study and their primary detection methods:

TABLE 2: DIMENSION C ANTI-PATTERN PENALTY SCHEDULE AND DETECTION METHODS

Anti-Pattern	Penalty	Primary Detection Source
Zombie Scrum	-15	Sprint review observation, GitHub (no increment), Discord engagement levels
No Real Definition of Done	-15	GitHub PRs, issue closure patterns, sprint review artifacts
Carryover Work	-10	GitHub issues, sprint-to-sprint comparison
Daily Scrum as Status Report	-10	Discord message patterns, directionality of communication
Product Owner as Project Manager	-10	Discord, task assignment patterns, role behavior
Overloaded Sprints	-10	Planned vs. completed story comparison, GitHub velocity
Multitasking Across Projects	-8	Discord context, GitHub commit patterns
ScrumBut Patterns	-8	Explicit deviations from Scrum events or artifacts

4.5.2 PENALTY WEIGHTING RATIONALE

High-severity anti-patterns (-15) are those that strike at the foundational purpose of Scrum itself. Zombie Scrum, a term coined by Russo and Verwijs (2021), describes teams that go through the motions of Scrum ceremonies without any of the intent, producing no real increment and experiencing no genuine improvement. It represents a complete failure of Agile integrity. The absence of a real Definition of Done is equally severe because it means the team has no shared standard for quality, making every "completed" item suspect and creating invisible technical debt.

Medium-severity anti-patterns (-10) represent significant but more targeted dysfunctions. Carryover work indicates persistent overcommitment or impediment mismanagement. The Daily Scrum as status report wastes a valuable synchronization event and signals that the team is not self-organizing. Product Owner as Project Manager conflates backlog management with task assignment, undermining developer autonomy. Overloaded sprints reflect a systemic failure of capacity planning that predictably produces incomplete work.

Low-severity anti-patterns (-8) represent real but more narrowly scoped dysfunctions. Multitasking across projects reduces focus and flow efficiency but may not prevent delivery if managed carefully. ScrumBut patterns, where teams

follow Scrum except for specific elements they find inconvenient, may reflect intentional adaptation but more often reflect resistance to practices that require effort or vulnerability.

4.6 AII SCORE CALCULATION

The following illustrates a complete AII calculation for a hypothetical sprint to demonstrate the model's application:

Dimension A. Value Alignment:

- Individuals & Interactions: 3
- Working Software: 4
- Customer Collaboration: 2
- Responding to Change: 3
- Average: 3.0 → **A Score = 3.0 × 20 = 60**

Dimension B. Scrum Practice Adherence:

- Sprint Planning: 4
- Daily Scrum: 2
- Sprint Review: 3
- Retrospective: 3
- Backlog Refinement: 3
- Definition of Done: 3
- Transparency/Inspection/Adaptation: 3
- Average: 3.0 → **B Score = 3.0 × 20 = 60**

Dimension C. Anti-Pattern Frequency:

- Carryover Work detected: -10
- Daily Scrum as Status Report detected: -10

- C Score = $100 - 20 = 80$

$$\text{All} = (0.25 \times 60) + (0.45 \times 60) + (0.30 \times 80) = 15 + 27 + 24 = 66$$

A score of 66 places this sprint in the "Developing" band (60–74), indicating a team that is following basic Scrum structure but has meaningful gaps in value alignment and at least two active anti-patterns requiring attention. Score band definitions are provided in Appendix D.

SECTION 5: DATA COLLECTION AND ANALYSIS

This section applies the All model systematically across all four ECU Pirate Forge teams. Analysis is organized by team, with each team's Dimension A, B, and C scores presented with supporting evidence before the composite All is calculated. ScrumPilot receives the most detailed treatment as the primary research subject. FormFlow, Pulse, and Sunflower Tracker are analyzed as comparative cases using all available data sources. All scores reference the rubric in [Appendix D](#). A corrected note on Sunflower Tracker's A Score: direct analysis of all four Dimension A values yields scores of 2 across all four values (average 2.0), producing an A Score of 40 rather than the 45 stated for Pulse. This distinction is reflected in the composite All calculations below.

5.1 SCRUMPILOT

ScrumPilot is an eight-member team building a lightweight Scrum management platform with embedded AI capabilities, designed as a focused alternative to tools like Jira. As the team I was embedded in throughout the semester, ScrumPilot benefits from the deepest available dataset, including burndown charts, retrospective documents, sprint summaries, and Discord activity across all four sprints.

Communication was genuinely multi-directional across the team. Multiple members initiated technical discussions, raised impediments, and coordinated asynchronously without reliance on a single coordinator. A PR tracking channel was established in Sprint 4 specifically to monitor review throughput, and pair programming sessions appear as early as Sprint 1. GitHub reinforces this pattern with 91 closed pull requests reflecting substantive multi-reviewer engagement. By Sprint 4, the application was deployed to Render using Docker containers and a persistent Postgres database. The main branch reflects 193 commits following intentional sprint-end promotion, consistent with the team's documented git workflow. Sprint 4 closed 26 of 27 committed PBIs, with the single carry-over representing a deliberate mid-sprint reprioritization. Sprint 3 velocity was recorded at 34 story points with a running average of 28 SP.

The most compelling Responding to Change evidence came in Sprint 4, when a planned Conversational AI feature was deprioritized mid-sprint in favor of cross-team recording and summarization capabilities benefiting all graduate students' research. This pivot was unanimously endorsed and documented. Prior sprints show consistent adaptation: the Craig bot limitation was acknowledged, a simpler workaround adopted, and the CI/CD pipeline improvements in Sprint 3 directly addressed Sprint 2 retrospective findings. The Sprint 4 ceremony restructuring, moving retro and planning to Friday, was itself a retro action.

Customer Collaboration is scored at 4 rather than 5. Professor Dietrick attended sprint reviews throughout the project, endorsed the mid-sprint bot pivot, and confirmed that ScrumPilot will be used for teaching in the next academic cohort. The team also proactively built out a Sprint 5 backlog for an incoming cohort. That said, the academic context structurally limits the feedback loop to one stakeholder rather than an external user population, which constrains how high this value can honestly score regardless of team behavior.

Dimension A scored 95. Individuals and Interactions 5, Working Software 5, Customer Collaboration 4, Responding to Change 5.

Dimension B scored 97. All seven practices scored 5 with one exception. Sprint Planning reflected dependency graphs published at sprint start, grooming preceding planning every sprint, and capacity planning with member constraints explicitly documented. Definition of Done is organized into five categories — Code Complete, Reviewed, Tested, Integrated, and Documented — enforced by the CI pipeline from Sprint 3 forward. Retrospectives produced documented action items with confirmed follow-through each sprint. Daily Scrum received a 4 rather than 5 because available evidence does not uniformly confirm peer-to-peer sprint goal orientation across all standup entries.

Dimension C scored 100. ScrumPilot received no anti-pattern penalties. All eight tracked anti-patterns were monitored and none met the evidence threshold in Appendix D. The most important clarification concerns carryover work. The sprint summaries list carry-over items each sprint, which superficially resembles the anti-pattern. On examination, each instance was a deliberate deprioritization with documented rationale and team consensus, not a failed commitment. Sprint 4's 26 of 27 completion rate with one intentional carry-over is the clearest evidence against applying this penalty. The Conversational AI PBI was subsequently completed.

All: $(0.25 \times 95) + (0.45 \times 97) + (0.30 \times 100) = 23.75 + 43.65 + 30.00 = 97$, Exemplary.

5.2 OBSERVED TEAM. AII ANALYSIS

All Result: 73 - Developing

FormFlow is a three-person team building a dynamic form generation application. Observing their sprint review and reading their Discord logs, the most striking characteristic of this team was the degree to which one person, referred to here as Developer A, carried the process. Developer A drove the sprint planning documentation, tracked PR chains, flagged blocked dependencies, held Developer B accountable in a direct but respectful Discord exchange the night before the sprint review, and served as the functional Scrum Master, coordinator, and primary communicator throughout the semester. The All correctly classifies this as a Hero Developer pattern, and it is worth naming it as a dysfunction, because a team that depends on one person for process integrity is fragile. If Developer A had been unavailable for Sprint 4, the blocked dependency chain would have had no one to manage it.

It is also worth acknowledging what Developer A's individual commitment actually produced: FormFlow had a deliverable. They closed 51 pull requests, maintained 348 commits to main, and demonstrated working software. The Hero Developer anti-pattern is real and harmful in the long run, but here it also demonstrates the value in practicing Scrum. When one team member genuinely internalizes Scrum values and practices them regardless of what others do, it creates a floor of quality the whole team benefits from. The recommendation for FormFlow is not to criticize what Developer A did, but to ask what would have happened if the other two team members had matched this approach. The gap between FormFlow's All of 73 and ScrumPilot's 97 is almost entirely explained by two people who did not. The ceiling for a team like FormFlow, if all three members had engaged at Developer A's level, is easily in the Proficient or Exemplary band.

The Customer Collaboration gap was the sharpest moment I observed during the FormFlow sprint review. One team member asked whether it would be possible to speak to the researchers who requested the application, explaining that he was having trouble visualizing the need for it. That question, raised four sprints into the project, reflects a lack of connection to the product vision, a direct example of what happens when sprint reviews become demos without context rather than working sessions with stakeholders who provide this sort of insight.

Dimension A scored 65. Individuals and Interactions received a 3, reflecting genuine peer accountability evident in Discord but with communication load concentrated in one member. Working Software received a 4, supported by 51 closed pull requests, 348 main branch commits, and working software demonstrated at each review. Customer Collaboration received a 3, though the product vision disconnect observed at the Sprint 4 review puts pressure on that score. Responding to Change received a 3, with Developer B's repeated PBI self-assignment pattern across two sprints without correction being the primary limiting factor.

Dimension B scored 71. Sprint Planning 4, Daily Scrum 4, Sprint Review 3, Retrospective 3, Backlog Refinement 4, Definition of Done 4, Transparency / Inspection / Adaptation 3. The Sprint Review score of 3 reflects the direct

observation of a review that lacked a sprint goal, a visible board, and PBI context before demos began. The professor had to ask which PBI was being demonstrated.

Dimension C scored 82. This reflects two detected anti-patterns. Carryover Work received a -10 penalty, confirmed across Sprints 2 through 4 with six explicit carry-over items documented in the Sprint 4 summary. Multitasking Across Projects received a -8 penalty, with one developer explicitly documenting military travel, a job offer process, and apartment hunting as competing demands during the sprint period.

All: $(0.25 \times 65) + (0.45 \times 71) + (0.30 \times 82) = 16.25 + 31.95 + 24.60 = 73$, Developing.

SUNFLOWER TRACKER

All Result: 47 - At Risk

Sunflower Tracker is a six-person team building a Webots simulation for solar panel sun-tracking. The defining finding for this team is the gap between work completed in feature branches and what actually reached the main branch. With 23 closed pull requests and legitimate simulation code in development, only 7 commits made it to main across four sprints. Their own git workflow explicitly required a dev-to-main promotion after every sprint review. It did not happen.

Dimension A scored 40. All four values were scored at 2, reflecting recurring attendance issues, no releasable main branch increment across the project, a skipped Sprint Review in Sprint 3, and the product vision disconnection described above.

Dimension B scored 34. Sprint Review received a 1, the lowest score in the cohort, because Sprint 3 was skipped entirely and the Sprint 4 review I observed had no Scrum board, no sprint goal, and no PBI context before demos began. Transparency/Inspection/Adaptation also received a 1, the only score consistent with a team that documented a sprint-end promotion requirement and did not follow it once. The Definition of Done scored a 2 rather than a 1 because the team wrote a thorough, Webots-specific DoD. The adherence to it was the problem, not the document itself.

Dimension C scored 72. This reflects three detected anti-patterns.

- Carryover Work received a -10 penalty, with Sprint 1 issues still open at Sprint 4.
- Overloaded Sprints received a -10 penalty, with Sprint 4 PRs blocked by missing unit tests.
- ScrumBut received an -8 penalty for the skipped Sprint 3 review with no documented rationale.

All: $(0.25 \times 40) + (0.45 \times 34) + (0.30 \times 72) = 10.00 + 15.30 + 21.60 = 47$, At Risk.

PULSE

All Result: 45 - At Risk

Pulse is a five-person team building a real-time classroom feedback platform. What makes this team the most instructive case in the cohort is not that they failed to document good process. They wrote the most detailed DoD and DoR of any team. What they could not do was follow it. The clearest signal came in Sprint 2, when the professor posted in Discord calling for drastically increased communication and attached grade consequences to get a response. These were primarily co-located students who shared class time. The problem was not logistics.

Dimension A scored 45. Working Software received a 3, the highest individual score for this team, reflecting 56 closed pull requests, 204 commits, and 181 passing tests. That number drops when you account for a broken endpoint at the Sprint 4 review and no functional UI until late in the semester. The remaining three values scored at 2. Engagement was professor-driven rather than team-initiated, the feedback loop was effectively the instructor rather than a collaborator, and the same workflow violations recurred across multiple sprints despite being corrected.

Dimension B scored 43. Nearly every practice scored at 2. Sprint Planning received a 2 because the professor created the repository, imported the PBIs, and wrote implementation steps in Sprint 1. Daily Scrum received a 2 because standups consisted of screenshots directed at the professor rather than peer coordination. The Definition of Done received a 2 despite being the most comprehensive in the cohort, because the professor was asking in PR reviews how he could verify that code met the DoD standard the team had written themselves. Backlog Refinement received a 3, the one relative strength, though that refinement was largely professor-owned.

Dimension C scored 47. This reflects five detected anti-patterns and the heaviest penalty total in the cohort. Zombie Scrum (-15) was applied because ceremonies existed but genuine team engagement required external coercion to sustain. Daily Scrum as Status Report (-10), Product Owner as Project Manager (-10), Carryover Work (-10), and ScrumBut (-8) each met their evidence thresholds based on Discord records and GitHub history.

All: $(0.25 \times 45) + (0.45 \times 43) + (0.30 \times 47) = 11.25 + 19.35 + 14.10 = 45$, At Risk.

5.3 SPRINT-BY-SPRINT ALL SCORES

[Table 3](#) presents the composite All scores and dimension breakdowns for all four teams across the observation period. Scores represent aggregate assessment across four sprints rather than per-sprint snapshots, reflecting the available data granularity.

The 50-point gap between ScrumPilot (97) and the lowest-scoring teams (45–47) is the study's most striking quantitative finding. It represents a team that embodied Agile integrity versus teams that possessed the documentation of process

without its behavioral substance. The FormFlow score of 73 occupies a meaningful middle position, genuine delivery with identifiable structural weaknesses, and provides the most instructive case for targeted improvement recommendations.

TABLE 3: COMPOSITE AII SCORES BY TEAM

Team	Dim A (25%)	Dim B (45%)	Dim C (30%)	All	Band
FormFlow	65	71	82	73	Developing
Pulse	45	43	47	45	At Risk
ScrumPilot	95	97	100	97	Exemplary
Sunflower Tracker	40	34	72	47	At Risk

SECTION 6: FINDINGS

The data produced some findings I expected and a few that surprised me. I will address each research sub-question directly, drawing directly from the AII scores from [Section 5](#).

6.1 VALUE ALIGNMENT CORRELATIONS

Of the four Agile values, Responding to Change showed the strongest relationship with positive outcomes in this cohort. ScrumPilot's score of 5 on this value was earned by observable, documented adaptation such as a mid-sprint feature pivot in Sprint 4, retrospective actions that actually changed subsequent sprint structure, and a team that treated the product backlog as a living document rather than a fixed commitment. The At Risk teams showed the inverse, the same problems reappearing sprint after sprint without adjustment. Pulse received repeated corrections about PR branch targets and continued sending PRs to main rather than dev across multiple sprints. Sunflower Tracker documented a git workflow requiring dev-to-main promotion after every sprint review and did not follow it a single time across four sprints. The data strongly suggests that Responding to Change is the Agile value with the most direct observable connection to delivery outcomes, and the one most easily verified through GitHub and Discord evidence.

Individuals and Interactions was the second strongest predictor. FormFlow's Hero Developer pattern is the clearest example of what happens when one person carries the interaction load. The team delivers, but the structural dependency creates fragility that any real disruption would expose. In this case, Developer A's availability held things together. If that had changed, there was no backup. ScrumPilot's cross-member engagement created the opposite condition. When individual members faced constraints across eight people, the team absorbed the impact without sprint failure because

no single person was the bottleneck. Customer Collaboration showed the weakest differentiation across the cohort, primarily because the academic context limits the depth of stakeholder relationships available regardless of team behavior. The most meaningful signal in this dimension came from its absence. A FormFlow team member said publicly during the sprint review that he did not really understand the need for the application and asked whether he could speak to the researchers who requested it. That moment stayed with me. Four sprints in, a team member had no connection to the product they had been building toward.

6.2 HIGH-IMPACT SCRUM PRACTICES

Among Scrum practices, Transparency, Inspection, and Adaptation had the strongest relationship with predictable delivery. That is not surprising. This practice essentially asks whether the team uses empirical evidence to guide its decisions. ScrumPilot published dependency graphs, tracked velocity, maintained a PR monitoring channel, and adjusted sprint ceremonies based on what they observed. Sunflower Tracker documented an identical requirement for dev-to-main promotion in their own git workflow and had 7 commits on main after four sprints. The gap between those two descriptions is the gap between a team that inspects and adapts and a team that does not.

The Definition of Done finding was the one that surprised me most, and I think it is the most instructive for future cohorts. Pulse wrote arguably the most detailed DoD in the cohort, more comprehensive than ScrumPilot's, more specific than FormFlow's, more domain-aware than Sunflower's. Yet their DoD adherence score was only a 2. The professor had to ask “how do I know that your code compiles and runs successfully? How do I know that your changes meet all acceptance criteria? How do I know that your changes meet DoD?” when there had been no test evidence or DoD verification. This was a team that had written a DoD covering frontend behavior, backend logic, database operations, mobile/desktop responsiveness, and real-time updates. Knowing what good looks like and practicing it are genuinely different things, and in the context of this dataset, the distinction is hard to ignore.

Sprint Review quality also showed a strong relationship with outcomes, which I observed directly for two teams. Neither the Sunflower Tracker review nor the FormFlow review opened with a sprint goal, displayed a Scrum board, or provided context before demoing. The professor had to ask which PBI a developer was working on during the FormFlow review. For anyone playing the role of a real stakeholder, both reviews would have been deeply frustrating. This isn't to say the work was bad, because the GitHub data suggests it was not, but because the absence of context made it impossible to evaluate whether it was the right work. Scrum's sprint review is specifically designed to generate that stakeholder feedback loop. Without the framing, it just becomes a demo.

6.3 MOST HARMFUL ANTI-PATTERNS

Zombie Scrum was the most damaging anti-pattern observed, and its presence in Pulse was identifiable before I looked at a single GitHub metric. The Discord log from Sprint 2 tells the story clearly: the professor, who was also functioning as the de facto Scrum Master and Product Owner, had to threaten grade consequences to get the team to check Discord once a day. These were primarily co-located students, four of five on campus during class time. The communication failure was not a remote work problem. The ceremonies existed. The repo existed. The DoD existed. But none of it was accompanied by genuine team engagement. There is no framework, Scrum or otherwise, that can compensate for this. The early warning signal for Zombie Scrum in this context is straightforward, if the volume and direction of Discord communication is driven by the instructor rather than the team, then the team is not self-organizing.

Carryover Work appeared in three of four teams and is the easiest anti-pattern to detect early. GitHub issue history across sprint boundaries is sufficient. The important distinction that the scoring model makes, and that I had to apply carefully to ScrumPilot, is between intentional deferral and failed commitment. ScrumPilot's sprint summaries list a carry-over item in Sprint 4, accompanied by a documented rationale in a conscious mid-sprint reprioritization. That is not carryover dysfunction, that is Responding to Change. The anti-pattern appears when stories simply reappear in the next sprint without comment, without adjustment, and without any indication the team reflected on why.

The Product Owner as Project Manager anti-pattern in Pulse deserves careful framing, because the professor's behavior was not wrong. It was a rational response to a team that was not functioning. When he wrote implementation steps, assigned PBIs, and directed technical decisions, he was doing what was necessary to get any output at all. That being said, that intervention prevented the team from developing the self-organization skills the course was designed to build. Teams do not become self-organizing while being managed. They need the discomfort of genuine accountability to develop that capacity. The All penalizes this pattern not to criticize the professor's judgment, but rather, because the behavioral outcome is measurably harmful to Agile integrity and needs to be noted with regard to their performance.

6.4 ALL AS A PREDICTIVE INDICATOR

The All demonstrates meaningful predictive validity within this dataset. The Exemplary team produced the strongest delivery evidence: a deployed application, 26 of 27 Sprint 4 PBIs completed, and professor confirmation that the product will be used in the next academic cohort. The Developing team produced consistent delivery with identifiable structural weaknesses. The At Risk teams both experienced delivery failures traceable to the specific anti-patterns and practice gaps their scores reflect. The composite nature of the index is what enables this predictive utility. Pulse's 56 closed PRs and 181 passing tests would suggest a healthier team if GitHub throughput were the only metric. The anti-pattern penalties and practice scores correctly weight the organizational dysfunction that those numbers conceal.

6.5 CROSS-TEAM COMPARISON

Three cross-team findings stand out as broadly significant. First, process documentation quality does not predict process adherence. Every team wrote thorough DoD, DoR, and git workflow documents. All four documented the same dev-to-main promotion requirement. The teams that followed their own documentation scored in the Exemplary and Developing bands. The teams that did not scored At Risk. This matters for how future cohorts invest their early sprint time. Writing better process documents will not produce better outcomes if the behavioral conditions to follow them are absent.

Second, team size did not explain performance. ScrumPilot had eight members and scored highest; FormFlow had three and scored second. The engagement quality of the individuals present mattered far more than headcount. Third, and most directly connected to the research question, the most meaningful differentiator between Exemplary and At Risk was whether teams changed their behavior based on what they observed. Scrum's empirical foundation, inspect and adapt, was validated positively in ScrumPilot and negatively in both At Risk teams. This is not a subtle finding. It is the entire premise of the framework confirmed by observable data.

SECTION 7: VISUALIZATION DASHBOARD

The visualization dashboard submitted alongside this paper presents the All findings across six interactive views, generated with AI assistance using only the hard data referenced throughout the research paper. The key visualizations are described below but are best interacted with via the dashboard [HERE](#).

7.1 ALL TREND LINE

The All Overview tab shows composite scores for all four teams: ScrumPilot (97, Exemplary), FormFlow (73, Developing), Pulse (45, At Risk), and Sunflower Tracker (47, At Risk). The 50-point gap between the highest and lowest scoring teams is the dashboard's most visually significant finding and the clearest illustration of the cohort's divergence.

7.2 ANTI-PATTERN HEATMAP

The Anti-Pattern Heatmap tab shows detected anti-patterns across all four teams. ScrumPilot shows no detections across all eight tracked patterns. Pulse shows the densest concentration, five anti-patterns including the two highest-penalty items (Zombie Scrum and PO as Project Manager). The visual contrast between ScrumPilot's empty row and Pulse's fully populated one communicates the study's core finding more immediately than any paragraph could.

7.3 VALUE-ALIGNMENT RADAR CHART

The Value Alignment tab displays an interactive radar chart. Clicking any team shows their shape across the four Agile values. ScrumPilot's near-perfect polygon contrasts sharply with the compressed, low-scoring shapes of the At Risk teams. Responding to Change and Individuals and Interactions show the widest separation between bands, confirming their role as the most differentiating values in this cohort.

7.4 SCRUM PRACTICE ADHERENCE SCORECARD

The Practice Adherence tab shows all seven Scrum practices across all four teams in both grouped bar and individual team views. Sprint Review and Transparency / Inspection / Adaptation show the widest variance, with ScrumPilot scoring 5 on both and Sunflower Tracker scoring 1 on both. This visual makes the case for those two practices as the highest-impact indicators of team health in this dataset.

7.5 ADDITIONAL VISUALIZATIONS

The Component Breakdown tab shows a stacked bar with each dimension's weighted contribution to the composite All. The All vs. GitHub scatter plot on the same tab is particularly instructive: Pulse appears productive by PR count alone (56 closed PRs), but its All of 45 reveals that throughput metrics cannot capture organizational dysfunction. The GitHub Activity tab documents the dev-to-main promotion gap. Sunflower's 7 main commits against their own documented requirement is visible immediately.

SECTION 8: RECOMMENDATIONS

The following recommendations are grounded in specific findings from the analysis rather than general Agile advice. Where a team is named, it is because the evidence is direct and teachable.

8.1 STRENGTHENING AGILE VALUE ALIGNMENT

Implement a mid-sprint health check at the sprint midpoint in which the team formally asks themselves an important question, is our current plan still the best path to the sprint goal? This addresses the Responding to Change weakness observed in Pulse and Sunflower Tracker, where teams waited until retrospectives, or longer, to surface needed adaptations. The FormFlow pattern of one developer's repeated PBI self-assignment behavior across two sprints without correction also illustrates the cost of delaying this conversation. A structured mid-sprint check creates a safe, scheduled

opportunity to surface concerns before they become sprint failures. An important note, this is a solution that introduces a new Ceremony in order to make this discussion a heavier priority. The ideal solution is that the team is taking this approach on a regular basis, each daily Scrum should answer this question, without needing an additional ceremony.

Require at least one external voice at every sprint review with a prepared question. The most significant Customer Collaboration failure I observed was a FormFlow team member who did not understand the purpose of their application. That is a direct consequence of holding four sprint reviews without genuine stakeholder engagement. Even in an academic context, a peer team, an instructor with a prepared question, or a future user persona can serve this function. The key is that the external voice comes with a question, forcing the team to treat the review as a working session rather than a showcase.

8.2 IMPROVING SCRUM PRACTICE EXECUTION

Require the sprint board to be displayed on screen to start each sprint review before any demo begins. Both of the sprint reviews I attended opened with individual demos and no context. The professor had to ask which PBI was being worked on. A visible board at the start of every review eliminates that problem entirely, forcing the team to begin with transparency about what the sprint contained, what was completed, and what was not.

Require the first agenda item of every retrospective to be a review of previous action items, with each marked done, in progress, or not started. The At Risk teams in this cohort showed recurring dysfunction across all four sprints with no evidence of retrospective follow-through. A retrospective that does not begin by asking whether the previous improvements were implemented will not produce behavioral change, it will produce a comfortable conversation that everyone forgets before the next ceremony.

8.3 ELIMINATING OR MITIGATING ANTI-PATTERNS

For Zombie Scrum prevention, I would recommend tracking Discord message volume as a leading indicator. In this cohort, a drop of more than 40% from the previous sprint's baseline, combined with standup entries directed at the instructor rather than the team, appeared before ceremony compliance failures became visible. The professor's Sprint 2 intervention in Pulse was the right response to an emerging pattern. Earlier detection would have enabled earlier intervention, before the dynamic became entrenched across three more sprints.

For Carryover Work reduction, establish a sprint commitment review at planning that requires the team to explicitly identify any story that is a dependency-chain risk, stories that will block other stories if not completed. These stories should be planned with a contingency: if the blocking story cannot be completed, what is the team's adaptation plan?

FormFlow's Sprint 4 blocked chain, a POST endpoint blocking three dependent PBIs, would have been partially mitigated by this planning discipline.

For Hero Developer mitigation, make sprint contribution distribution visible by team member at each sprint review. If any single member accounts for more than 50% of closed PBIs in a sprint, discuss it explicitly in the retrospective as a structural risk rather than celebrating it as individual achievement. Developer A's contribution pattern in FormFlow was strong evidence of a high performer, but it was also evidence of a structural dependency that would create a capacity crisis if that person were unavailable.

8.4 ENHANCING PREDICTABILITY AND QUALITY

Enforce dev→main promotion at every sprint review as a non-negotiable sprint completion criterion. All four teams documented this requirement in their git workflows, but not all participants followed it consistently. Making it a graded sprint completion criterion, not merely a recommended practice, would create the accountability needed for teams like Sunflower Tracker that documented the process but did not follow it across four sprints. The Scrum Guide's statement that the increment "must meet the Definition of Done" and must be "releasable" connects directly to this discipline.

Add CI/CD pipeline setup as a Sprint 1 or Sprint 2 mandatory deliverable, not an optional future enhancement.

ScrumPilot's CI/CD pipeline, established in Sprint 3, enforced build quality and test coverage automatically, providing objective DoD verification that removed the subjectivity from PR approvals. FormFlow's Sprint 1 sprint summary notes "CI/CD pipeline placeholders, currently non-functional". A functional pipeline from Sprint 2 onward would have provided earlier quality enforcement and likely reduced the carryover accumulation observed in Sprint 4.

SECTION 9: REFLECTION

9.1 SURPRISES AND UNEXPECTED FINDINGS

The finding that surprised me most was the inverse relationship between process documentation quality and adherence in Pulse. I expected the weakest documents to correspond with the weakest adherence. Pulse had the most detailed DoD in the cohort and the lowest Dimension B score. The professor had to ask how he could know their code met the Definition of Done in PR reviews, for a team that had written a DoD covering frontend behavior, backend logic, database operations, mobile responsiveness, and real-time updates. That gap forced me to think carefully about what process documentation actually does and does not do. It does not create accountability. It does not generate engagement. Those things come from culture and from individuals who have genuinely bought in. A document is not a substitute for either.

9.2 LIMITATIONS OF THE AII MODEL

The AII model, as designed, carries several inherent constraints worth acknowledging. First, the aggregate scoring approach produces a single score per team across the full observation period rather than per-sprint scores. A team that struggled early and improved significantly by Sprint 4 looks identical in the composite to one that was consistently mediocre throughout. Longitudinal per-sprint scoring would tell a more complete story but requires more disciplined data collection than was feasible in this study. Second, the sample size of four teams across four sprints provides directional evidence but not statistical confidence. The correlations observed between AII scores and team outcomes are meaningful within this cohort but cannot be generalized without a significantly larger dataset across multiple semesters and team compositions. Third, the academic context creates a structural mismatch between what the model assumes and what the environment produces. Scrum assumes clearly defined roles: a Product Owner who owns the vision, a Scrum Master who serves the team, and Developers who execute. In a class setting, students fill all three roles simultaneously with no formal training in any of them. The model is not wrong to flag the resulting dysfunction, but it is worth acknowledging that it is a demanding tool applied to a population that was never operating under ideal conditions.

9.3 LIMITATIONS OF THE APPLIED RESEARCH

Beyond the model itself, this particular application of the AII has several constraints specific to how the data was collected and analyzed. The most significant is information asymmetry across teams. ScrumPilot published burndown charts, retrospective documents, and sprint summaries in shared Discord channels throughout the semester, giving me a depth of visibility into my own team that I simply did not have for the others. It is possible that equivalent transparency from the other teams would have raised or lowered their scores in ways that cannot be known. Second, sprint review observation was limited to a single review per team at the end of the semester. A different review week could have produced different findings. One strong or weak showing does not necessarily represent a team's typical performance, and another researcher attending the same review might have walked away with modestly different evaluations even using the same rubric. Third, without access to a live Scrum board for the peer teams, I could not speak confidently to the timing of work within sprints. Whether tasks sat untouched until the final days, sat in review for extended periods, or moved steadily from start to finish is invisible in the data I had access to. The burndown pattern matters for evaluating sprint health and its absence is a real gap. Finally, as a member of one of the evaluated teams, personal bias cannot be fully eliminated despite the deliberate choice to rely on hard data and AI-assisted scoring. The methodology mitigates it, but cannot remove it entirely.

9.4 PERSONAL TAKEAWAYS

I work as a member of Scrum projects daily in my work as a Software Developer. I am very familiar with the process and intend to become a Certified Scrum Developer later this year. So going into this I already had an expectation of what Scrum and Agile methodologies offer, as well as their pros and cons. Still, I walked away with some surprising takeaways.

Before this course, one of my critiques has been the number of ceremonies and the amount of time sunk into them. I often find myself criticizing the practices because the large amount of time committed to these ceremonies on a weekly basis can be better utilized if I were free to continue writing code, tests, and documentation. However, studying the difference between teams who embraced the process the way my team did in ScrumPilot and comparing it to the results of teams who did not fully commit to Scrum, it becomes abundantly clear that the benefits outweigh my perceived drawbacks. The ceremonies are of the utmost importance, and this means simply going through the motions is not enough.

I've come to feel that Scrum is more than the set of practices a team follows, it's a set of commitments that require a certain team culture to function. The ceremonies are visible, the culture is not, but the culture is what determines whether the ceremonies do anything useful. What I take from this into professional practice is something like a diagnostic instinct. When I join or observe a Scrum team, I will not look to velocity or burndown charts as the main metric of success. I will look at who participates properly in standup, whether anyone raises an impediment, whether the sprint review informs the audience of the breadth of the Sprint, and whether the retrospective references what the team had previously committed to changing. Those behaviors are observable early, and as this data suggests, highly predictive.

SECTION 10: CONCLUDING THOUGHTS

METHODOLOGY BIAS AND LIMITATIONS

I took every measure available to me as a researcher to mitigate personal bias. The analysis was grounded in hard data using AI-assisted scoring to root all analysis purely in fact. However, there is an inherent informational advantage in my own team's scores that no scoring model would fully eliminate here. ScrumPilot published burndown charts, velocity data, dependency graphs, retrospective documents, and attendance records throughout the semester as deliverables for our weekly submissions. That breadth of documentation gave the scoring model a fuller picture than was available for any other team. It is entirely possible that equivalent access to those artifacts for FormFlow, Pulse, or Sunflower Tracker would have shifted their scores in either direction. This may have revealed strengths that Discord and GitHub alone could not capture, or exposed additional gaps. The 50-point gap between ScrumPilot and the At Risk teams is supported by the available evidence, but it should be read with the awareness that the evidence itself was not equally distributed across teams.

CONCLUSION

The central research question this paper set out to answer was how Agile Manifesto values, Scrum best-practice adherence, and anti-pattern presence collectively influence team outcomes. The answer, supported by data from four teams across four sprints, is straightforward: They completely determine them. Every team in this cohort started with the same tools, the same course requirements, and nearly identical process documentation. The divergence in outcomes was entirely a function of how genuinely each team practiced what they had written down.

That is not a surprising finding, but rather a confirmation. The Agile Manifesto has existed since 2001. The Scrum framework is well defined, extensively documented, and backed by decades of practitioner experience. The AI did not uncover something hidden, it measured what Scrum was always designed to require and produced a score that reflected how well each team met that standard. The metric works because it is rooted in a methodology that works. Agile is not theoretical, it is proven. However, it only works when it is actually followed.

What this cohort illustrates, and what makes the findings particularly instructive, is how difficult genuine adoption is without the traditional structure that makes Scrum effective. These teams had no dedicated Product Owner holding a product vision, no Scrum Master focused on team health and impediment removal, and no external stakeholder providing consistent feedback. Students filled these roles simultaneously while also doing the development work. Under those conditions, the teams that found ways to self-organize, maintain their own accountability, and genuinely embrace the ceremonies rather than just perform them were able to navigate that structural gap. The teams that could not, were exposed by it.

The difference in outcomes between the Exemplary and At Risk bands in this study was not talent, tooling, or luck. It was discipline and genuine commitment to the process. That is the most transferable lesson this research produces, and it is the reason a measurement framework like the AI has value beyond any single cohort. As long as Agile methodology remains the standard for software development teams, the question of whether a team is doing Scrum or truly practicing it will remain worth asking. This paper is one attempt to answer it rigorously.

REFERENCES

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. <https://agilemanifesto.org/>
- Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum: The rules of the game. <https://scrumguides.org/scrum-guide.html>
- Russo, D., & Verwijs, C. (2021). The Zombie Scrum survival guide. Addison-Wesley Professional.
- Kniberg, H., & Ivarsson, A. (2012). Scaling Agile @ Spotify with tribes, squads, chapters and guilds. Spotify. <https://blog.crisp.se/wp-content/uploads/2012/11/SpotifyScaling.pdf>
- Edmondson, A. (1999). Psychological safety and learning behavior in work teams. *Administrative Science Quarterly*, 44(2), 350–383. <https://doi.org/10.2307/2666999>
- Duhigg, C. (2016, February 25). What Google learned from its quest to build the perfect team. *The New York Times Magazine*. <https://www.nytimes.com/2016/02/28/magazine/what-google-learned-from-its-quest-to-build-the-perfect-team.html>
- Dikert, K., Paasivaara, M., & Lassenius, C. (2016). Challenges and success factors for large-scale agile transformations: A systematic literature review. *Journal of Systems and Software*, 119, 87–108. <https://doi.org/10.1016/j.jss.2016.06.013>
- Brown, W. H., Malveau, R. C., McCormick, H. W., & Mowbray, T. J. (1998). *AntiPatterns: Refactoring software, architectures, and projects in crisis*. Wiley.
- Koenig, A. (1995). Patterns and antipatterns. *Journal of Object-Oriented Programming*, 8(1), 46–48.

APPENDIX A: AGILE MANIFESTO

A.1 ORIGINS AND CONTEXT

The Agile Manifesto was authored in February 2001 by seventeen software practitioners who gathered in Snowbird, Utah to discuss lightweight development methods. The signatories, including Kent Beck, Martin Fowler, Ken Schwaber, Jeff Sutherland, and others, came from diverse methodological backgrounds but shared a common frustration with the heavyweight, documentation-driven processes that dominated software development at the time. The result of their meeting was a brief but consequential document: four value statements and twelve supporting principles that have since reshaped how software is built across the industry.

It is important to understand what the Manifesto is and is not. It is not a methodology. It does not prescribe specific practices, ceremonies, or artifacts. It is a philosophical orientation, a set of priorities intended to guide decision-making when teams face trade-offs. The practices that emerged from it, including Scrum, Extreme Programming, and Kanban, are implementations of its spirit rather than derivations of its text.

A.2 THE FOUR VALUES

The Manifesto centers on four value statements, each expressed as a preference between two legitimate goods rather than an absolute rejection of the lesser-valued item. The full text reads:

"We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value: (Beck et al., 2001)

Individuals and interactions over processes and tools Working software over comprehensive documentation Customer collaboration over contract negotiation Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more. " (Beck et al., 2001)

The final caveat is critical and frequently overlooked. The Manifesto does not argue that processes, documentation, contracts, or plans have no value. It argues that when a conflict or trade-off arises between the two sides, teams should resolve it in favor of the left-hand item. A team that produces no documentation is not "more Agile", it is simply undisciplined. A team that chooses a working conversation over a rigid process requirement when they are in tension is embodying the value correctly.

Each value has specific implications for team behavior that inform the All Dimension A scoring model (see [Section 4.3](#)).

VALUE 1: INDIVIDUALS AND INTERACTIONS OVER PROCESSES AND TOOLS

This value holds that the quality of human collaboration is more determinative of project success than the sophistication of the processes or tools used. Teams that over-index on tools, using Jira configuration as a substitute for genuine coordination, or enforcing rigid process gates that slow rather than enable delivery, violate this value even when they believe they are being rigorous.

In practice, this value manifests as teams that prioritize direct conversation over ticket updates, that resolve ambiguity through discussion rather than documentation, and that adapt their use of tools to serve the team rather than constraining the team to serve the tool. Communication patterns, collaboration breadth, and the degree to which team members proactively engage with each other are all behavioral indicators of this value.

VALUE 2: WORKING SOFTWARE OVER COMPREHENSIVE DOCUMENTATION

This value establishes working software as the primary measure of progress. In pre-Agile environments, teams frequently spent months producing requirement specifications, design documents, and project plans before a single line of functional code was written, often discovering at delivery that what was built did not match what was needed. This value reorients the team's sense of progress around demonstrable, functional increments.

"Working software" does not mean software without documentation. It means that when a team must choose between writing more documentation and producing more working functionality, they should favor the latter. In modern practice, this extends to any working product increment, not exclusively software in the narrow technical sense. The key behavioral indicator is whether the team can demonstrate something real at the end of each sprint.

VALUE 3: CUSTOMER COLLABORATION OVER CONTRACT NEGOTIATION

This value reflects a fundamental shift in how teams relate to the people they build software for. Contract negotiation implies a transactional relationship in which requirements are fixed at the outset, responsibilities are clearly partitioned, and disputes are resolved by reference to the original agreement. Customer collaboration implies an ongoing partnership in which requirements evolve through dialogue and the customer is an active participant in shaping the product throughout development.

For student teams, this value is interpreted broadly. The "customer" may be an instructor, a simulated stakeholder, or the team's own shared vision of the product. The behavioral indicator is whether feedback, from whatever source, actively shapes the backlog and the team's direction, or whether it is acknowledged and then set aside.

VALUE 4: RESPONDING TO CHANGE OVER FOLLOWING A PLAN

This value acknowledges that software development is a complex, knowledge-generating activity in which early assumptions are frequently wrong. Plans are necessary, they establish direction and enable coordination, but teams must be willing to revise them when new information makes the original plan obsolete.

Teams that violate this value often do so with good intentions. Staying on plan feels like discipline; changing the plan feels like failure. The Manifesto inverts this framing. A team that adapts its plan in response to what it learns is demonstrating good judgment, not admitting defeat. The behavioral indicator is whether the team treats the sprint backlog and roadmap as living documents that respond to reality, or as commitments that must be honored regardless of what is discovered.

A.3 THE TWELVE PRINCIPLES

The Manifesto is accompanied by twelve principles that elaborate on the four values and provide more concrete guidance for their application. These principles are grouped thematically below for clarity.

DELIVERY AND VALUE

1. *"Our highest priority is to satisfy the customer through early and continuous delivery of valuable software"* (Beck et al., 2001). Delivery should begin early and continue without long gaps. Value is the measure of what is delivered, not volume or effort.
2. *"Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale"* (Beck et al., 2001). Short delivery cycles reduce risk and accelerate feedback. The sprint in Scrum is the direct implementation of this principle.
3. *"Working software is the primary measure of progress"* (Beck et al., 2001). This principle reinforces Value 2. Progress reports, burndown charts, and story point counts are useful signals, but the only true measure of progress is software that functions.

EMBRACING CHANGE

4. *"Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage."* Change is not a problem to be managed, it is a source of value when handled well. Teams should design their processes to accommodate late-arriving information. (Beck et al., 2001)

5. "Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely." Sustainable pace is both a productivity principle and a human one. Heroic sprints and crunch culture are anti-Agile regardless of short-term output. (Beck et al., 2001)

COLLABORATION AND COMMUNICATION

6. "Business people and developers must work together daily throughout the project." Collaboration is not a phase, it is a continuous activity. Handoffs between business and development are a source of waste and misalignment. (Beck et al., 2001)
7. "The most efficient and effective method of conveying information to and within a development team is face-to-face conversation." While remote-first work has challenged the literal application of this principle, its intent, that rich, synchronous communication outperforms asynchronous documentation, remains valid and relevant. (Beck et al., 2001)
8. "Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done." Self-organization requires trust. Micromanagement, even when well-intentioned, undermines the motivation and ownership that Agile teams depend on. (Beck et al., 2001)

TECHNICAL EXCELLENCE

9. "Continuous attention to technical excellence and good design enhances agility." Technical debt is an agility tax. Teams that allow quality to erode in the name of speed eventually find that the debt slows them more than the shortcuts saved. This principle connects directly to the Definition of Done and to the Working Software value. (Beck et al., 2001)
10. "Simplicity, the art of maximizing the amount of work not done, is essential." Teams should resist the urge to build more than is currently needed. Over-engineering and gold-plating consume capacity that could be directed toward higher-priority value. (Beck et al., 2001)

TEAM DYNAMICS AND IMPROVEMENT

11. "The best architectures, requirements, and designs emerge from self-organizing teams." Imposed architecture and top-down design produce solutions that fit the organization chart rather than the problem. Self-organizing teams, given clear goals and appropriate constraints, consistently produce more adaptive solutions. (Beck et al., 2001)

12. "At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly." This principle is the philosophical foundation of the Sprint Retrospective. Continuous improvement is not optional, it is a core Agile commitment. (Beck et al., 2001)

A.4 APPLICATION TO THE AII

The four Agile values form the basis of Dimension A of the Agile Integrity Index. Each value is operationalized through observable behavioral indicators drawn from Discord communication patterns, GitHub activity, and sprint ceremony observation. The scoring rubric for Dimension A (see [Appendix D](#)) defines five performance levels for each value, anchored to specific behaviors that a researcher can observe and score consistently.

The twelve principles serve a supporting role in this study. They are not independently scored but are referenced throughout the analysis as interpretive guides when evaluating ambiguous evidence. For example, Principle 5 (sustainable pace) informs the assessment of overloaded sprints in Dimension C, and Principle 12 (regular reflection) informs the scoring of Sprint Retrospectives in Dimension B.

APPENDIX B: SCRUM BEST PRACTICES

B.1 OVERVIEW OF THE SCRUM FRAMEWORK

Scrum is an empirical framework for developing, delivering, and sustaining complex products. First formalized by Ken Schwaber and Jeff Sutherland in the mid-1990s and codified in the Scrum Guide, most recently updated in November 2020, Scrum is not a methodology in the prescriptive sense. It does not define engineering techniques, specify tooling, or dictate how work should be technically executed. Instead, it provides a lightweight structure of accountabilities, events, and artifacts that create the conditions for teams to apply judgment, inspect outcomes, and adapt continuously.

The framework rests on three pillars of empiricism: **transparency**, **inspection**, and **adaptation**. Transparency requires that the process and its outputs be visible to those performing the work and those receiving it. Inspection requires that the artifacts and progress toward goals be examined frequently and honestly. Adaptation requires that when inspection reveals deviation from acceptable outcomes, adjustments be made as quickly as possible. These three pillars are not independent, without transparency, inspection is uninformed; without inspection, adaptation is arbitrary; without adaptation, transparency and inspection are academic exercises.

Scrum is intentionally incomplete. The framework deliberately leaves room for teams to determine the specific practices, tools, and techniques that work best in their context. This is a feature, not a limitation. The best practices described in this

appendix represent the collective wisdom of practitioners, researchers, and the Scrum Guide itself on how to implement the framework in a way that realizes its intent rather than merely satisfying its letter.

B.2 SCRUM VALUES

The 2020 Scrum Guide elevated the five Scrum values to a position of central importance within the framework. These values are not aspirational additions, they are described as foundational to the trust and collaboration that make Scrum work. When teams embody these values, the three empirical pillars become natural outcomes rather than enforced requirements.

B.2.1 COMMITMENT

Commitment in Scrum does not mean promising to complete a fixed scope regardless of what is discovered. It means committing to the sprint goal, to continuous improvement, and to each other as a team. This distinction is important: a team that commits to a sprint goal while remaining open to adjusting the plan in service of that goal is practicing Scrum correctly. A team pressured to commit to a specific list of stories and held accountable for delivering every one of them regardless of circumstances is being subjected to a misapplication of this value.

Best practice requires that commitment be visible and shared. The sprint goal should be articulated clearly so that every team member understands what success looks like for the sprint. Commitment to improvement should be evidenced by retrospective action items that are taken seriously and followed through.

B.2.2 FOCUS

Focus requires that the team concentrate its energy on the work of the sprint and the sprint goal above all other competing demands. Context switching, multitasking across projects, and mid-sprint scope additions all violate this value by fragmenting team attention and reducing the quality and speed of delivery.

Best practice requires that the sprint backlog represent a realistic and bounded scope of work, that team members are protected from external interruptions during the sprint where possible, and that the Daily Scrum specifically addresses progress toward the sprint goal rather than general task status.

B.2.3 OPENNESS

Openness requires that the team and its stakeholders be honest about the work, the challenges encountered, and the team's progress. This value creates the conditions for effective inspection, if team members conceal impediments, misrepresent progress, or avoid difficult conversations, the empirical foundation of Scrum collapses.

Best practice requires that impediments be surfaced quickly rather than absorbed silently, that sprint reviews honestly represent the state of the increment rather than presenting a curated showcase, and that retrospectives create genuine space for team members to identify what is not working without fear of judgment or reprisal.

B.2.4 RESPECT

Respect requires that team members treat each other as capable, well-intentioned professionals. In a self-organizing team, respect is the social contract that makes autonomy sustainable. Without it, conflict goes unresolved, feedback is avoided, and the team fragments rather than coheres.

Best practice requires that different skill sets and perspectives be genuinely valued, that disagreements be resolved through dialogue rather than hierarchy, and that no single team member's contributions be consistently dismissed or overlooked. In the context of this study, respect is observable in the inclusivity of Discord conversations and the breadth of engagement in GitHub code reviews.

B.2.5 COURAGE

Courage requires that team members have the willingness to do the right thing even when it is difficult, to raise impediments even when doing so is uncomfortable, to push back on unrealistic expectations, to admit uncertainty about estimates, and to conduct honest retrospectives that address real issues rather than surface-level symptoms.

Best practice requires that the Scrum Master actively create an environment where courage is safe to exercise, that the team is not penalized for raising concerns, and that the sprint review is an honest assessment of done and not-done rather than a performance designed to manage stakeholder perception.

B.3 SCRUM ACCOUNTABILITIES

The 2020 Scrum Guide replaced the term "roles" with "accountabilities" to emphasize that Scrum defines responsibilities rather than job titles (Schwaber & Sutherland, 2020). The three accountabilities. Product Owner, Scrum Master, and Developers, together form the Scrum Team, which is collectively responsible for all product-related activities.

B.3.1 THE PRODUCT OWNER

The Product Owner is accountable for maximizing the value of the product resulting from the Scrum Team's work. This accountability encompasses the entire product backlog, its content, ordering, and transparency. The Product Owner is not a requirements writer or a project manager. They are a value strategist whose primary job is to ensure the team is always working on the highest-value items available.

Maintaining a product backlog that is ordered, transparent, and sufficiently refined so that the top items are ready for sprint selection. "Ready" means that stories are well-understood, appropriately sized, and accompanied by acceptance criteria that the Developers and the Product Owner agree on.

Being available to the Developers throughout the sprint to answer questions, clarify requirements, and make prioritization decisions quickly. A Product Owner who is inaccessible forces developers to make assumptions, which introduces rework risk and misalignment.

B.3.2 THE SCRUM MASTER

The Scrum Master is accountable for establishing Scrum as defined in the Scrum Guide and for the team's effectiveness. This is fundamentally a service role, the Scrum Master serves the Developers, the Product Owner, and the organization, but does not direct the work of any of them.

Coaching the team in self-management and cross-functionality rather than directing or assigning work. The Scrum Master's goal is to make themselves progressively less necessary by building the team's capacity to operate effectively without intervention.

Facilitating Scrum events when needed, not necessarily running every meeting, but ensuring that each event achieves its intended purpose within its timebox. A skilled Scrum Master knows when to facilitate actively and when to let the team self-organize.

B.3.3 THE DEVELOPERS

In Scrum, "Developers" refers to all team members who are committed to creating any aspect of a usable increment each sprint, not exclusively software engineers (Schwaber & Sutherland, 2020). The Developers are collectively accountable for creating a plan for the sprint (the sprint backlog), instilling quality by adhering to the Definition of Done, adapting their plan each day toward the sprint goal, and holding each other accountable as professionals.

Self-organizing around the work rather than waiting for task assignments. The sprint backlog is the Developers' responsibility to manage. They pull work based on priority and their own assessment of who is best positioned to do it, rather than having work assigned to them by the Scrum Master or Product Owner.

Maintaining a cross-functional orientation in which the team collectively possesses all the skills needed to create a done increment. While individual specializations are natural and valuable, best practice requires that knowledge-sharing, pair work, and collaborative problem-solving prevent the formation of impenetrable silos.

B.4 SCRUM EVENTS

Scrum defines five events. Each event is timeboxed, it has a maximum duration, and each serves a specific purpose within the empirical cycle of inspect and adapt. Skipping events, shortening them arbitrarily, or conducting them without genuine engagement degrades the team's ability to inspect and adapt effectively.

B.4.1 THE SPRINT

The Sprint is the heartbeat of Scrum, a fixed-length event of one month or less during which the Scrum Team creates a done, usable increment toward the product goal. All other Scrum events occur within the sprint. The sprint's fixed cadence creates the rhythm that makes empiricism possible: regular, bounded periods of work followed by inspection and adaptation.

Maintaining a consistent sprint length. Varying sprint length disrupts the team's ability to establish velocity baselines, complicates retrospective comparisons, and introduces uncertainty into stakeholder planning. Most effective teams settle on one or two-week sprints.

Treating the sprint goal as inviolable even when the sprint backlog changes. If conditions change mid-sprint such that some sprint backlog items can no longer be completed, the team should renegotiate scope with the Product Owner, but the sprint goal itself should remain the target unless extraordinary circumstances make it genuinely unachievable.

B.4.2 SPRINT PLANNING

Sprint Planning initiates the sprint by establishing what can be delivered and how the work will be achieved. The Scrum Guide recommends a timebox of eight hours for a one-month sprint, proportionally shorter for shorter sprints. For a two-week sprint, four hours is a reasonable target.

Defining a meaningful sprint goal before selecting backlog items. The sprint goal provides the "why" of the sprint and enables the team to make coherent trade-off decisions if things change mid-sprint. A sprint goal that is simply a list of items to complete ("complete stories 14, 15, and 16") is not a goal, it is a task list.

Selecting backlog items that the Developers have confidence they can complete given their current capacity. Capacity planning should account for team member availability, known commitments, and a realistic assessment of velocity based on recent sprints rather than theoretical maximum throughput.

B.4.3 DAILY SCRUM

The Daily Scrum is a 15-minute event held every working day of the sprint, at the same time and place, for the Developers. Its purpose is to inspect progress toward the sprint goal and adapt the sprint backlog as necessary. Despite its brevity, the Daily Scrum is one of the most frequently misapplied Scrum events.

Focusing the discussion on progress toward the sprint goal rather than individual task status. The classic three questions, "What did I do yesterday? What will I do today? Are there any impediments?", are a useful starting point but should serve as a lens on the sprint goal, not a ritual recitation of activity (Schwaber & Sutherland, 2020).

Keeping the event timeboxed to 15 minutes. Detailed problem-solving discussions that arise during the Daily Scrum should be taken offline to a follow-up conversation with only the relevant parties, rather than consuming the attention of the entire team.

B.4.4 SPRINT REVIEW

The Sprint Review is held at the end of the sprint to inspect the increment and adapt the product backlog. It is a working session, not a presentation. The Scrum Guide timeboxes it to four hours for a one-month sprint. Its most important output is an updated product backlog that reflects what was learned during the review.

Demonstrating only work that meets the Definition of Done. Items that are incomplete, partially working, or do not meet quality criteria should not be presented as done. Presenting undone work as done, even with caveats, erodes trust, creates a misleading picture of progress, and undermines the integrity of the increment.

Inviting genuine stakeholder participation and feedback rather than conducting a one-way demonstration. The Sprint Review is the team's primary opportunity to course-correct based on real feedback. A review in which stakeholders are passive observers produces none of this value.

B.4.5 SPRINT RETROSPECTIVE

The Sprint Retrospective concludes the sprint and provides the Scrum Team with a dedicated opportunity to inspect how they worked and identify improvements for the next sprint. It is timeboxed to three hours for a one-month sprint. The Scrum Guide identifies it as the team's primary mechanism for continuous process improvement.

Creating an environment of psychological safety in which team members can raise genuine concerns without fear of judgment, blame, or consequences. Retrospectives that consistently produce only positive observations or surface-level commentary are not functioning as designed, they reflect a safety problem, not a team that has nothing to improve.

Producing a small number of specific, actionable improvement items rather than an exhaustive list of everything that could be better. Two or three improvements that are actually implemented produce more value than ten improvements that are forgotten by the following Monday.

B.4.6 BACKLOG REFINEMENT

Backlog Refinement is not formally defined as a Scrum event in the Scrum Guide, it is described as an ongoing activity that occurs throughout the sprint. However, most experienced practitioners treat it as a semi-regular structured session because its outcomes are foundational to effective Sprint Planning.

Refining backlog items on a rolling basis so that the top items are always in a "ready" state before sprint planning. A commonly used readiness criterion is the INVEST model: stories should be Independent, Negotiable, Valuable, Estimable, Small, and Testable.

Including the Developers in refinement discussions, not just the Product Owner. Developers bring technical perspective that is essential for accurate estimation, identification of dependencies, and meaningful acceptance criteria. Refinement conducted exclusively by the Product Owner produces stories that may be well-prioritized but are poorly understood.

B.5 SCRUM ARTIFACTS

Scrum defines three artifacts, each of which includes a commitment that defines the artifact's purpose and provides a standard against which it can be evaluated.

B.5.1 PRODUCT BACKLOG

The Product Backlog is an ordered list of everything that might be needed in the product. It is the single source of work for the Scrum Team and is the Product Owner's primary accountability. The commitment associated with the Product Backlog is the **Product Goal**, a description of the future state of the product that serves as a long-term objective.

Maintaining a backlog that is ordered by value, not by chronology or arbitrary priority labels. The ordering should reflect the Product Owner's best current judgment about which items will deliver the most value given current knowledge, stakeholder needs, and strategic context.

Keeping the backlog visible and accessible to all team members and relevant stakeholders. A backlog that exists only in the Product Owner's mind or in a private document fails the transparency pillar of Scrum.

B.5.2 SPRINT BACKLOG

The Sprint Backlog consists of the sprint goal, the backlog items selected for the sprint, and the plan for delivering them. It is owned by the Developers and represents their best current understanding of the work required to achieve the sprint goal. The commitment associated with the Sprint Backlog is the **Sprint Goal**.

Treating the sprint backlog as a living document that is updated daily as the Developers learn more about the work. A sprint backlog that does not change between planning and review either reflects a perfectly predictable sprint, which is rare, or a team that is not inspecting and adapting at the task level.

Making the sprint backlog visible to the entire Scrum Team and relevant stakeholders. A physical or digital board that shows the state of all sprint items at any given moment is the most common implementation. Items should move through defined states, typically To Do, In Progress, and Done, based on their actual status.

B.5.3 INCREMENT

The Increment is the sum of all completed product backlog items from the current sprint and all previous sprints. It must meet the Definition of Done to be considered part of the increment, and it should be usable, meaning that it could theoretically be released whether or not the Product Owner decides to do so. The commitment associated with the Increment is the **Definition of Done**.

Producing at least one done increment per sprint. If no increment is produced, the sprint has failed in its most fundamental purpose. Teams that consistently produce incomplete increments need to address the root cause, whether overcommitment, impediments, technical debt, or unclear acceptance criteria, rather than normalizing the pattern.

Treating the Definition of Done as a quality floor, not a ceiling. Stories that meet the DoD are done; stories that do not, regardless of how much work has been invested in them, are not. This binary clarity is essential for maintaining the integrity of the increment and preventing the accumulation of hidden technical debt.

B.5.4 DEFINITION OF DONE

The Definition of Done is a formal description of the state a product backlog item must achieve to be considered complete. It is shared across the Scrum Team and, where applicable, aligned with organizational standards. A strong Definition of Done is one of the most powerful quality management tools available to a Scrum Team.

Defining the DoD as a team during the early sprints and revisiting it regularly in retrospectives. The DoD should evolve as the team's quality standards mature and as technical practices improve.

Including quality criteria in the DoD, not only task completion indicators. A DoD that consists solely of "code committed" or "story closed" provides no quality assurance. A strong DoD includes criteria such as code reviewed, unit tests passing, acceptance criteria verified, and documentation updated where applicable.

APPENDIX C: COMPREHENSIVE SCRUM ANTI-PATTERN LIST

C.1 INTRODUCTION TO SCRUM ANTI-PATTERNS

The term "anti-pattern" originates in software engineering, first popularized by Andrew Koenig in 1995 and later expanded by Brown et al. in *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis* (1998). An anti-pattern is distinguished from a simple mistake by two characteristics: it appears to be a reasonable or even intentional solution, and it consistently produces negative outcomes that are worse than having no solution at all. Anti-patterns are insidious precisely because they are not obviously wrong, they often feel like pragmatic adaptations, reasonable compromises, or sensible shortcuts.

In the Scrum context, anti-patterns have been catalogued by a range of practitioners and researchers. Christiaan Verwijs and Johannes Russo's work on Zombie Scrum, Craig Larman and Bas Vodde's writings on fake Agile, Linda Rising and Mary Lynn Manns' work on organizational patterns, and a growing body of empirical research on Scrum failure modes all contribute to the taxonomy presented here. This appendix organizes Scrum anti-patterns into five categories: process anti-patterns, role anti-patterns, communication anti-patterns, planning anti-patterns, and cultural anti-patterns. (Russo & Verwijs, 2021)

For each anti-pattern, this appendix provides a definition, a description of how it typically manifests, its root causes, its consequences for team outcomes, and early warning signals that a team or observer can use to detect it before it becomes entrenched. Anti-patterns included in the All Dimension C scoring model are marked with their corresponding penalty weight.

C.2 PROCESS ANTI-PATTERNS

Process anti-patterns are dysfunctions in how the team executes the structural elements of Scrum, its events, cadence, and artifacts. They are often the most visible category of anti-pattern because they produce observable deviations from prescribed Scrum practice.

C.2.1 ZOMBIE SCRUM (*All PENALTY: -15*)

Definition: Zombie Scrum, a term coined by Russo and Verwijs (2021) in *The Zombie Scrum Survival Guide*, describes a state in which a team performs all of the ceremonies and structural elements of Scrum without any of its intent. The team holds standups, plans sprints, and conducts retrospectives, but produces no real increment, experiences no genuine improvement, and feels no connection between their work and meaningful outcomes.

Manifestation: Zombie Scrum teams are identifiable by a pervasive sense of going through the motions. Sprint reviews demonstrate incomplete or non-functional work, or no work at all. Retrospectives produce action items that are never implemented. Sprint goals exist on paper but bear no relationship to the team's actual daily activity. Stakeholders are largely absent from reviews. The team expresses low engagement, low energy, and a fatalistic acceptance that Scrum "doesn't really work here." (Russo & Verwijs, 2021)

Root Causes: Zombie Scrum typically emerges when Scrum is imposed on a team without organizational support or genuine buy-in, when the team has no connection to the value their product creates, when management continues to operate in a command-and-control mode that contradicts Scrum's self-organization principles, or when the team has been so thoroughly burned by failed change initiatives that they comply superficially to avoid conflict. (Russo & Verwijs, 2021)

Consequences: Zombie Scrum is arguably the most damaging anti-pattern in this taxonomy because it combines all of the overhead of Scrum with none of its benefits. Teams in this state waste ceremony time, accumulate technical debt, produce no meaningful increment, and gradually demoralize team members who came to Scrum hoping for genuine improvement. The pattern also tends to self-reinforcing, the longer it persists, the harder it becomes to reignite genuine engagement. (Russo & Verwijs, 2021)

Early Warning Signals: Declining attendance or energy at ceremonies; sprint reviews with nothing to demonstrate; retrospective action items that repeat without resolution; team members who cannot articulate the sprint goal; absence of stakeholder engagement; flat or declining velocity combined with no process improvement conversations.

C.2.2 NO REAL DEFINITION OF DONE (AII PENALTY: -15)

Definition: This anti-pattern occurs when a team either has no formal Definition of Done, or has one that is so minimal or inconsistently applied that it provides no meaningful quality assurance. Items are closed based on subjective judgment or social pressure rather than objective criteria.

Manifestation: Stories are marked complete when development is finished but testing has not occurred, acceptance criteria have not been verified, or code has not been reviewed. The Definition of Done may exist as a document but is not referenced during sprint activities or checked during sprint reviews. Different team members apply different standards to story completion, creating an inconsistent and unreliable increment.

Root Causes: Teams frequently skip or underinvest in their Definition of Done because establishing and maintaining it requires effort and creates accountability. When organizational pressure to show progress is high, a loose or absent DoD allows teams to report completion of work that is not genuinely done. In student contexts, the DoD is often left vague because the team has not yet encountered the downstream costs of technical debt and inconsistent quality.

Consequences: Without a meaningful DoD, the increment is unreliable. Technical debt accumulates invisibly. Sprint Reviews lose their validity because "done" means something different to everyone. Stakeholders receive a distorted picture of progress. The team spends increasing amounts of time revisiting work that was marked complete but was not truly finished, eroding both velocity and trust.

Early Warning Signals: Stories reopened after closure; defects discovered in items marked done; sprint reviews that reveal incomplete functionality; inconsistent quality between items delivered by different team members; inability to articulate what "done" means when asked directly.

C.2.3 CARRYOVER WORK (AII PENALTY: -10)

Definition: Carryover occurs when product backlog items selected for a sprint are not completed by the end of that sprint and are moved into the next sprint. A single instance of carryover may reflect normal uncertainty; a persistent pattern of carryover indicates a systemic problem.

Manifestation: The same stories appear in multiple consecutive sprints. Sprint velocity is consistently lower than planned. The sprint backlog at the end of the sprint regularly contains items in "in progress" status. Sprint reviews acknowledge that items were not completed but commit to finishing them "next sprint", a commitment that is frequently repeated.

Root Causes: Carryover typically results from overcommitment during sprint planning, stories that are larger or more complex than estimated, unresolved impediments that block progress, or mid-sprint scope additions that displace planned work. It may also reflect a team culture in which "starting" work is equated with "progressing" work, allowing items to remain in-flight indefinitely without approaching completion.

Consequences: Persistent carryover erodes sprint predictability, makes velocity an unreliable planning metric, and creates a false sense of progress. It also contributes to an overloaded sprint in the following period as carryover items compete with newly planned work for team attention.

Early Warning Signals: Stories that span multiple sprints; velocity significantly below planned capacity; sprint backlog items in "in progress" state at sprint end; teams that treat carryover as normal rather than exceptional; declining stakeholder confidence in delivery estimates.

C.2.4 SKIPPED OR PERFUNCTORY CEREMONIES

Definition: This anti-pattern occurs when Scrum events are skipped entirely, conducted so briefly that they cannot serve their purpose, or executed as empty rituals without genuine engagement.

Early Warning Signals: Calendar patterns showing irregular or cancelled ceremonies; ceremony durations significantly below recommended timeboxes; team members who cannot recall the last retrospective; sprint reviews held without a demonstration; Daily Scrums held only a few times per sprint.

C.2.5 SCRUMBUT (All PENALTY: -8)

Definition: ScrumBut is a term used to describe situations in which a team claims to practice Scrum but omits or modifies specific elements they find inconvenient. The pattern typically takes the form: "We do Scrum, but [we don't do retrospectives / we don't have a sprint goal / we don't timebox our standups]."

Manifestation: The team identifies as Agile and Scrum-practicing but has carved out specific exceptions to the framework that consistently exempt them from its most demanding or vulnerable aspects. These exceptions are often framed as adaptations to their unique context rather than acknowledged as deviations.

Root Causes: ScrumBut often reflects resistance to practices that require effort, vulnerability, or organizational change. Retrospectives require psychological safety. Sprint goals require the Product Owner to make hard prioritization decisions. Timeboxing requires discipline. When these requirements feel uncomfortable, teams find reasons to exempt themselves while maintaining the identity of "doing Scrum." (Edmondson, 1999)

Consequences: The Scrum framework is designed as an integrated whole, its elements reinforce each other, and removing any one of them degrades the whole. ScrumBut teams often experience the costs of Scrum's overhead without accessing its benefits, leading them to conclude that Scrum doesn't work rather than recognizing that a partial Scrum is producing partial results.

Early Warning Signals: Team members who describe their process with frequent qualifications; ceremonies that are consistently skipped for the same stated reason; absence of specific Scrum artifacts like the sprint goal or DoD; team statements like "we do Agile our own way."

C.2.6 FAKE RETROSPECTIVES

Definition: This anti-pattern describes retrospectives that occur on schedule and produce action items, but do so in a way that avoids genuine reflection on real problems. The team discusses only comfortable topics, agrees on surface-level improvements, and never addresses the underlying dysfunctions that are limiting their effectiveness.

Early Warning Signals: Short retrospectives with no difficult conversations; action items that repeat without resolution; reluctance to discuss specific incidents; facilitators dominating the session; absence of disagreement or tension; team members who cannot recall specific retrospective outcomes.

C.2.7 OVERLOADED SPRINTS (*All PENALTY: -10*)

Definition: Overloaded sprints occur when the team commits to more work during sprint planning than their actual capacity can support. This results in predictable underdelivery, carryover, and the normalization of a gap between planned and completed work.

Manifestation: Sprint velocity consistently falls below planned capacity. The team regularly carries items over. Sprint reviews frequently acknowledge incomplete work. Team members report working extended hours to try to meet sprint commitments. The gap between planned and completed stories is treated as a normal condition rather than a problem requiring root cause analysis.

Root Causes: Overloaded sprints often result from external pressure to commit to more than is realistic, optimistic estimation without reference to historical velocity, failure to account for non-development activities such as meetings, code reviews, and administrative work, or a team culture in which saying "we can't commit to that much" feels like failure.

Consequences: Overloaded sprints are self-defeating. The pressure they create reduces quality, increases defect rates, accelerates burnout, and produces the carryover patterns that further disrupt subsequent sprint planning. Stakeholders receive an inflated picture of planned delivery that is consistently not achieved, eroding confidence in the team's predictability.

Early Warning Signals: Consistent gap between planned and completed velocity; stories regularly carrying over; team members reporting time pressure; sprint planning sessions where capacity is not explicitly discussed; stakeholders expressing surprise at sprint review about what was not completed.

C.3 ROLE ANTI-PATTERNS

Role anti-patterns emerge when the Scrum accountabilities are misunderstood, blurred, or actively violated. They frequently reflect organizational structures or individual behaviors that are incompatible with Scrum's self-organization principles.

C.3.1 PRODUCT OWNER AS PROJECT MANAGER (*ALL PENALTY: -10*)

Definition: This anti-pattern occurs when the Product Owner conflates their accountability for the product backlog with the authority to direct how the Developers do their work. Rather than managing value and priority, they manage tasks, timelines, and individual assignments.

Manifestation: The Product Owner assigns specific tasks to specific developers rather than allowing the team to self-organize around sprint backlog items. They dictate how work should be technically implemented rather than what outcomes should be achieved. They manage the sprint backlog, which belongs to the Developers, rather than the product backlog. They produce status reports based on individual developer activity and use these to evaluate performance.

Root Causes: This anti-pattern is common when a Product Owner has a project management background and defaults to familiar behaviors. It is also common in organizations where someone with project manager authority has been reassigned to the Product Owner role without a genuine shift in their responsibilities or mindset. Organizational structures that reward delivery control over value delivery reinforce this pattern.

Consequences: When the Product Owner manages tasks rather than value, Developers lose autonomy, ownership, and motivation. The self-organization that Scrum depends on is replaced by dependency on direction from above. The Product Owner also loses effectiveness in their actual accountability, the product backlog, because their attention is consumed by sprint-level task management.

Early Warning Signals: Product Owner directly assigning stories to individual developers; Product Owner attending Daily Scrum and directing the conversation; Developer questions about work directed to the Product Owner rather than resolved by the team; sprint backlog managed by the Product Owner rather than the Developers.

C.3.2 ABSENT PRODUCT OWNER

Definition: This anti-pattern describes a Product Owner who is inaccessible to the Scrum Team during the sprint, unavailable to answer questions, clarify requirements, or make prioritization decisions in a timely manner.

Early Warning Signals: Developers making requirement decisions independently; sprint planning conducted without meaningful Product Owner input; acceptance criteria missing or added retroactively; stories requiring re-work after review; long response times to Developers' backlog questions.

C.3.3 SCRUM MASTER AS NOTE-TAKER OR ADMINISTRATOR

Definition: This anti-pattern occurs when the Scrum Master's primary activities are administrative, scheduling meetings, taking minutes, managing tools, and producing status reports, rather than coaching, facilitating, and removing impediments.

Early Warning Signals: Scrum Master's primary output is meeting notes or status reports; impediments unresolved across multiple sprints; no evidence of coaching conversations; Scrum Master unable to articulate recent process improvements they have driven.

C.3.4 DEVELOPER SILOS

Definition: Developer silos occur when team members work exclusively within their own area of specialization rather than collaborating across the full scope of sprint work. Rather than a cross-functional team that collectively owns the increment, the team fragments into specialists who protect their domain and avoid work outside it.

Early Warning Signals: Work consistently distributed along specialization lines; team members who cannot describe what others are working on; pull requests reviewed only by subject matter experts; developers expressing reluctance to work on unfamiliar story types.

C.3.5 HERO DEVELOPER

Definition: The Hero Developer anti-pattern describes a situation in which one team member consistently saves the sprint through heroic individual effort, working excessive hours, solving crises single-handedly, and carrying the team through delivery. The hero is celebrated rather than the pattern being recognized as a systemic problem.

Early Warning Signals: Sprint success consistently attributed to one individual; other team members expressing dependency rather than ownership; one team member's commits representing a disproportionate share of total work; team member working hours significantly above others.

C.4 COMMUNICATION ANTI-PATTERNS

Communication anti-patterns describe dysfunctions in how information flows within the team and between the team and its stakeholders. They are particularly relevant to the AI given the primary reliance on Discord as a data source.

C.4.1 DAILY SCRUM AS STATUS REPORT (*AI PENALTY: -10*)

Definition: This anti-pattern occurs when the Daily Scrum transforms from a peer-to-peer coordination event into a status reporting meeting where team members report their activity to the Scrum Master, Product Owner, or manager.

Manifestation: Team members address their updates to the Scrum Master or a senior team member rather than to each other. The content focuses on what was done rather than progress toward the sprint goal. Impediments are rarely raised. The Scrum Master or manager dominates the conversation with follow-up questions. Team members experience the event as a performance review rather than a coordination opportunity.

Root Causes: This pattern frequently reflects a fundamental misunderstanding of the Daily Scrum's purpose by the Scrum Master or organizational leadership. When a manager or authority figure attends the Daily Scrum and asks status questions, team members naturally orient their communication toward that person. The event becomes a status meeting by behavioral gravity even without explicit intent.

Consequences: The Daily Scrum loses its value as a self-organization tool. Team members learn to prepare a daily status update rather than genuinely coordinating with each other. Impediments are suppressed because raising them in a status meeting feels like admitting failure. The team's ability to adapt the sprint backlog daily is compromised.

Early Warning Signals: Updates addressed to the Scrum Master rather than the team; no discussion of sprint goal progress; impediments rarely raised during the event; event consistently finishing in under five minutes with no substantive discussion; Scrum Master asking follow-up questions of individual developers.

C.4.2 IMPEDIMENT HOARDING

Definition: Impediment hoarding occurs when team members consistently absorb blockers silently rather than surfacing them in the Daily Scrum or to the Scrum Master. Impediments are worked around individually rather than resolved systemically.

Early Warning Signals: Zero impediments raised across multiple sprints; stories blocked in GitHub with no corresponding discussion; retrospectives revealing sprint-time blockers that were not raised during the sprint; Scrum Master unable to identify impediments they removed during the sprint.

C.4.3 HIPPO-DRIVEN BACKLOG

Definition: HiPPO stands for Highest Paid Person's Opinion. This anti-pattern describes a product backlog whose ordering is driven by the preferences of the most senior or influential person in the room rather than by evidence, user feedback, or strategic value assessment.

Early Warning Signals: Backlog reprioritization following executive meetings without stakeholder feedback data; Product Owner unable to articulate the rationale for current backlog ordering; team members expressing cynicism about backlog priorities; user research or feedback not visible in backlog decisions.

C.5 PLANNING ANTI-PATTERNS

Planning anti-patterns are dysfunctions in how the team estimates, selects, and commits to work. They undermine sprint predictability and contribute to the delivery failures that erode stakeholder confidence.

C.5.1 VELOCITY GAMING

Definition: Velocity gaming occurs when a team manipulates its reported velocity, typically by inflating story point estimates, to create the appearance of higher throughput, meet external targets, or reduce pressure to commit to more work.

Early Warning Signals: Story point estimates increasing without corresponding complexity changes; velocity increasing while stakeholder satisfaction decreases; estimation discussions focused on targets rather than effort; reluctance to decompose stories because larger items inflate velocity more.

C.5.2 NO SPRINT GOAL

Definition: This anti-pattern describes sprints in which the team selects a set of backlog items to complete but does not articulate a coherent sprint goal, a statement of what the sprint is intended to achieve and why.

Early Warning Signals: Sprint goals that are simply lists of stories; team members unable to articulate the sprint goal without referencing the backlog; sprint reviews focused on item completion rather than goal achievement; no discussion of the sprint goal during the Daily Scrum.

C.5.3 ESTIMATION ANCHORING

Definition: Estimation anchoring occurs when early estimates, whether from a subject matter expert, a previous similar story, or an authority figure, anchor the team's collective estimation rather than allowing genuine independent judgment by all estimators.

Early Warning Signals: Estimates converging immediately on the first number offered; brief estimation discussions without genuine exploration; senior developers' estimates adopted without challenge; estimation retrospectives revealing consistent over or underestimation.

C.5.4 SPRINT ZERO ABUSE

Definition: Sprint Zero refers to an initial sprint used to establish infrastructure, environments, and foundations before feature delivery begins. This becomes an anti-pattern when it extends beyond its legitimate purpose, becoming an indefinite preparation phase that delays the delivery of working software.

Early Warning Signals: Sprint Zero extending beyond one or two sprints; inability to demonstrate any user-facing functionality; infrastructure work expanding to fill available sprint capacity; stakeholders expressing impatience with lack of visible progress.

C.6 CULTURAL ANTI-PATTERNS

Cultural anti-patterns are the most difficult to detect and address because they are embedded in team norms, organizational expectations, and individual behaviors rather than in specific ceremonies or artifacts. They often underlie multiple other anti-patterns simultaneously.

C.6.1 MULTITASKING ACROSS PROJECTS (*All PENALTY: -8*)

Definition: This anti-pattern occurs when team members are simultaneously assigned to multiple projects, products, or teams, requiring them to context-switch regularly and preventing the focused attention that effective sprint work requires.

Manifestation: Team members reference work on other projects during the Daily Scrum. Commits and PR activity in GitHub show irregular patterns consistent with interrupted work cycles. Team members are unavailable for collaboration because they are in meetings or working on other products. Sprint capacity planning is complicated by unpredictable availability caused by external demands.

Root Causes: Multitasking assignments typically reflect organizational resource management practices that treat developers as fungible resources to be allocated across multiple demands simultaneously. The hidden cost of context switching, estimated by some researchers at 20–40% of productive time per additional project, is frequently underestimated or ignored by management.

Consequences: Context switching reduces the quality and speed of work on all projects simultaneously. Team members cannot achieve the sustained focus that complex problem-solving requires. Sprint planning becomes unreliable because capacity is unpredictable. Team cohesion suffers because members are not fully present to the team's work.

Early Warning Signals: Developers referencing other project work in standups; irregular commit patterns suggesting interrupted work cycles; availability gaps during the sprint; sprint capacity consistently lower than nominal headcount would suggest.

C.6.2 FORCED AGILE

Definition: Forced Agile describes an organizational context in which Scrum has been mandated by leadership without genuine understanding of or commitment to what it requires. The organization adopts the vocabulary and ceremonies of Scrum while maintaining the management behaviors, incentive structures, and cultural norms of its previous approach.

Early Warning Signals: Scrum ceremonies held alongside traditional status meetings; fixed scope commitments required alongside sprint planning; management tracking individual developer task completion rather than team outcomes; team members expressing cynicism about Agile in retrospective discussions.

C.6.3 LACK OF PSYCHOLOGICAL SAFETY

Definition: Psychological safety, defined by Amy Edmondson as the shared belief that the team is safe for interpersonal risk-taking, is a prerequisite for effective Scrum rather than a nice-to-have cultural feature. Its absence is an anti-pattern because without it, virtually every other Scrum practice is compromised. (Edmondson, 1999)

Early Warning Signals: Silent retrospectives; unanimous agreement in all team discussions; absence of productive conflict; team members expressing concerns privately that they did not raise in ceremonies; individuals deferring consistently to the most senior voice in the room.

C.6.4 BLAME CULTURE

Definition: Blame culture describes a team environment in which mistakes, failures, and missed commitments are met with attribution of individual fault rather than systemic inquiry. Rather than asking "what in our process allowed this to happen," the question becomes "who is responsible for this."

Early Warning Signals: Defensive communication patterns in retrospectives; individuals quick to explain why something was not their responsibility; absence of team-level accountability language ("we" vs. "he/she"); low retrospective participation from team members who were involved in sprint failures.

C.6.5 MANAGEMENT BY JIRA

Definition: Management by Jira describes a situation in which leadership evaluates team and individual performance primarily through tool metrics, tickets closed, story points completed, time logged, rather than through actual outcomes, product quality, or team health.

Early Warning Signals: Leadership referencing Jira data instead of sprint review outcomes; individual performance discussions centering on ticket metrics; teams optimizing story point values for velocity appearance; discrepancy between tool metrics and actual product quality or stakeholder satisfaction.

C.7 ANTI-PATTERN SUMMARY TABLE

The following table provides a consolidated reference for all anti-patterns included in the All Dimension C scoring model, along with those tracked qualitatively throughout the analysis.

Anti-Pattern	Category	All Penalty	Primary Detection Source
Zombie Scrum	Process	-15	Observation, Discord, GitHub
No Real Definition of Done	Process	-15	GitHub, Sprint Review
Carryover Work	Process	-10	GitHub Issues, Sprint Comparison
Overloaded Sprints	Process	-10	Velocity Data, GitHub
Daily Scrum as Status Report	Communication	-10	Discord Message Patterns
Product Owner as Project Manager	Role	-10	Discord, Assignment Patterns
Multitasking Across Projects	Cultural	-8	Discord, GitHub Commit Patterns
ScrumBut Patterns	Process	-8	Ceremony Observation, Discord
Skipped or Perfunctory Ceremonies	Process	Qualitative	Calendar, Discord
Fake Retrospectives	Process	Qualitative	Discord, Observation
Absent Product Owner	Role	Qualitative	Discord Response Patterns
Scrum Master as Note-Taker	Role	Qualitative	Observation, Discord
Developer Silos	Role	Qualitative	GitHub, Discord
Hero Developer	Role	Qualitative	GitHub Commits, Discord
Impediment Hoarding	Communication	Qualitative	Discord, GitHub
HiPPO-Driven Backlog	Communication	Qualitative	Discord, Backlog History
Velocity Gaming	Planning	Qualitative	Velocity Trend, GitHub
No Sprint Goal	Planning	Qualitative	Sprint Artifacts, Discord
Estimation Anchoring	Planning	Qualitative	Planning Observation
Sprint Zero Abuse	Planning	Qualitative	GitHub, Sprint History
Forced Agile	Cultural	Qualitative	Observation, Discord
Lack of Psychological Safety	Cultural	Qualitative	Observation, Retrospective Quality
Blame Culture	Cultural	Qualitative	Discord, Retrospective Observation
Management by Jira	Cultural	Qualitative	Leadership Behavior, Tool Metrics

APPENDIX D: ALL SCORING RUBRIC

DIMENSION A RUBRIC: AGILE MANIFESTO VALUE ALIGNMENT

Score 5. Exemplary: Frequent multi-directional Discord communication across all members; substantive PR reviews with multiple reviewers; every sprint produces a deployable increment meeting all DoD criteria; stakeholder actively shapes backlog and feedback is integrated within the sprint; team adapts plan proactively and retrospective actions are consistently implemented the following sprint.

Score 1. Absent: Communication essentially absent; members working in isolation; no working increment produced; no stakeholder engagement; no adaptation to change; no retro or retro produces no follow-through.

Score 2. Minimal: Minimal cross-member communication with one or two members dominating; perfunctory PR reviews; incomplete or non-demonstrable increments; stakeholder rarely engaged; team resists mid-sprint changes and retro actions are rarely implemented.

Score 3. Developing: Moderate communication with some members consistently quiet; reviews present but shallow; increment produced with some gaps and DoD partially applied; stakeholder present but passive; team reacts to change but does not anticipate it and some retro actions are completed.

Score 4. Proficient: Regular communication with most members engaged; PR reviews present and mostly substantive; increment produced most sprints with minor DoD gaps; stakeholder attends reviews and feedback is mostly acted on; team adjusts when clearly needed and most retro actions are followed through.

DIMENSION B RUBRIC: SCRUM BEST-PRACTICE ADHERENCE

Score 5. Exemplary: Sprint goal defined and referenced throughout sprint; daily standups held every working day with peer-to-peer coordination focused on sprint goal; working software demonstrated at review with active stakeholder feedback; retrospective held each sprint with specific actions and prior actions closed; regular refinement sessions with INVEST-ready stories; DoD written with quality criteria and consistently referenced; board current, metrics visible, behavioral changes made based on observation.

Score 1. Absent: No planning evidence; Daily Scrum not held or conducted as pure status meeting; sprint review not held; retrospective not held; no refinement activity; no DoD; no transparency artifacts.

Score 2. Minimal: No sprint goal; standups 1–2 times per week with status reporting to SM or instructor; no demo at review; retrospective rarely held with no real action items; rare or absent refinement; DoD minimal or not referenced; board frequently outdated with no adaptation.

Score 3. Developing: Sprint goal vague or absent; standups 3–4 times per week with some status reporting; partial demo at review with passive stakeholders; retrospective held inconsistently with generic action items; infrequent refinement with stories sometimes unprepared; DoD exists but inconsistently applied; board sometimes outdated with limited adaptation.

Score 4. Proficient: Sprint goal defined with most stories estimated; standups near-daily and mostly peer-to-peer; demo occurs at review with some stakeholder engagement; retrospective most sprints with action items and some follow-through; occasional refinement with most stories ready; DoD exists and is mostly applied; board mostly current with some metrics tracked.

DIMENSION C RUBRIC: ANTI-PATTERN FREQUENCY

Zombie Scrum: –15. Detection, sprint review observation, Discord engagement levels, GitHub increment evidence.

Threshold, no working increment produced AND ceremonies held without genuine team engagement (external coercion required to sustain ceremony participation).

ScrumBut Patterns: –8. Detection, ceremony observation, Discord records, sprint artifacts. Threshold, explicit documented deviation from prescribed Scrum practice with no rationale for the adaptation recorded.

Multitasking Across Projects: –8. Detection. Discord explicit references to other project work, GitHub commit pattern irregularities. Threshold, explicit documented evidence of concurrent project obligations fragmenting capacity during the sprint period.

Overloaded Sprints: –10. Detection, planned vs. completed story ratio, velocity data. Threshold, sprint completion rate below 70% of planned capacity; pattern observed in at least two sprints.

Product Owner as Project Manager: –10. Detection. Discord task assignment patterns, sprint backlog ownership.

Threshold. PO directly assigning stories to individual developers or directing task-level implementation decisions across multiple sprints.

Daily Scrum as Status Report: –10. Detection, Discord message directionality and content analysis. Threshold, updates consistently addressed to instructor or SM rather than to team peers; sprint goal not referenced in standup content across multiple sprints.

Carryover Work: –10. Detection, GitHub issue history across sprint boundaries. Threshold, two or more stories carry over from committed sprint scope; pattern repeats across at least two sprints; carryover is not accompanied by documented rationale for intentional deferral.

No Real Definition of Done: –15. Detection. GitHub PR patterns, story closure evidence, sprint review artifacts. Threshold. DoD absent, undocumented, or stories consistently closed without meeting written quality criteria.

AII COMPOSITE SCORE INTERPRETATION

90–100 Exemplary: Team fully embodies Agile values in observable behavior, executes Scrum events with high consistency and genuine intent, and operates free of significant anti-patterns. Delivery is predictable, quality is consistent, and the team demonstrates continuous improvement across sprints.

Below 45 Critical: Fundamental Agile integrity issues across all three dimensions. The team is practicing Scrum in name only. Structured coaching, external facilitation, or significant team restructuring is required before improvement is likely.

45–59 At Risk: Significant dysfunction present in multiple dimensions. Anti-patterns are likely active and practice adherence is inconsistent. Delivery is unpredictable and the team requires structured process intervention.

60–74 Developing: Moderate performance with meaningful gaps in at least one dimension. The team follows basic Scrum structure but has areas where ceremony quality, value alignment, or anti-pattern management require targeted attention.

75–89 Proficient: Strong overall health with minor gaps in one or two dimensions. The team practices Scrum with genuine intent but has identifiable areas for improvement. Delivery is generally consistent with occasional disruptions.